

基于 STM32N647 的智能元器件仓储系统与端侧 AI 视觉调度优化

摘要

本项目设计并实现了一套基于 STM32N647 主控芯片与 FreeRTOS 实时操作系统的智能元器件仓储系统与端侧 AI 视觉调度平台。该系统以高校实验室及电子研发团队的硬件开发环境为核心应用场景，面向物料种类繁多检索困难、传统 BOM 清单人工核对低效、以及多用户共用仓储缺乏权限管理等痛点，提供精准化、智能化、闭环式的元器件存储与空间调度管理方案。

系统集成了 MIPI 图像采集、二维码扫码、热敏打印机、SK9822 阵列灯珠、电磁锁与电磁铁、VL53L1 测距及 AHT10 温湿度等多种控制与传感组件，构建出一个多通道、高效率的数字化仓储控制系统。通过 ESP32-C3-MINI-1 模块，系统可与微信小程序实时通信，一键导入 BOM 清单并驱动出入库精准空间寻址，形成闭环式物料数据管理流程。用户可通过 4.3 寸 RGB 触摸屏进行本地交互，并通过语音助手、蜂鸣器与热敏打印获得实时反馈。

在系统架构方面，软件层采用模块化设计，结合 FreeRTOS 实现多任务调度与边缘离线人脸识别，提升系统实时性与可维护性；硬件层则以主控板为核心，辅以分区模块驱动、4 路独立输出供电、1 路输入充电链路与电磁防护电路构成完整系统，具备良好的抗干扰与可靠性能。为提高系统稳定性与用户体验，项目还结合了微型试管与抽屉空间的立体机械防呆结构、图形界面（GUI）及电源按键控制的智能低功耗机制。

在项目实施过程中，团队对本地边缘 AI 视觉、高密度灯带状态机调度、二维码扫码解码等关键模块进行了深度优化与调试，解决了包括大功率感性器件引起的 200V 反向电动势浪涌抑制、低功耗状态机平滑切换等多个工程难题，充分体现了系统集成与复杂问题解决能力。

本系统在保障研发资产安全、消除传统物料管理“糊涂账”、大幅提升硬件团队物料调度效率方面具有重要应用价值，亦具备良好的扩展性与应用前景。

第一部分 作品概述

1.1 功能与特性

本系统基于 STM32N647 主控芯片与 FreeRTOS 实时操作系统，构建了一个多通道、智能化、端侧边缘 AI 驱动的元器件仓储与视觉调度平台。系统支持 N6 本地端侧离线 AI 人脸鉴权解锁、元器件可视化管理、BOM 清单一键自动比对、物料空间高精寻址和元器件标签热敏打印，实现研发实验室元器件管理的数字化、闭环式与规范化管理。

系统利用 MIPI 图像采集协同电磁锁实现安全身份校验，并通过二维码扫码识别录入物料，联动 SK9822 灯珠阵列与推拉电磁铁达成秒级仓位指示与抽屉自动弹出，结合微型试管与抽屉空间的立体机械防呆结构避免物料错取误放。用户界面采用 4.3 寸 RGB 触摸屏进行直观交互，具备语音提示与热敏打印即时反馈功能。系统通过 ESP32-C3 无线通信与微信小程序连接，基于 MQTT 协议实时同步库存状态，实现多端协同调度与动态预警。

系统支持多任务调度与四路独立供电轨架构，并且配备电源路径管理可实现锂电池单独供电以及 typec 边充边用，具备良好的实时性、电气可靠性与可拓展性，满足硬件开发实验室对资产精细化管理的多样化需求。

1.2 应用领域

本系统面向高校科研实验室与电子研发团队开发场景，主要应用于元器件物资的精细化仓储管理和 BOM 清单的智能视觉调度分析。对于高校电子设计竞赛备赛、日常实验教学以及多人共用实验室等高频物料周转情境，系统可作为端侧智能化工具，对贴片阻容、高价值芯片等繁杂元器件的存量、封装及耐压参数进行精准记录与空间状态指引（抽屉弹出/试管亮灯），帮助研发团队消除物料“糊涂账”，从底层避免错取误放，大幅提升研发调度效率。系统通过无线网络连接微信小程序，将出入库明细与权限流水实时同步至移动端，为管理人员提供动态库存预警与一键 BOM 比对，便于长期研发资产的追溯与合规审计。同时，该系统也适用于中小型科技企业物料房、高职院校电子教研室等对物料有安全管控需求的群体。凭借其本地离线边缘 AI 识别、四路供电高可靠性、强多模态交互和立体机械防呆特性，该系统可广泛应用于智能仓储物流终端、数字化实验室管理

系统、工业精密物资管控柜等多个领域，为用户提供便捷、科学的物资闭环监控解决方案。

1.3 主要技术特点

本系统以 STM32N647 主控芯片为核心，结合 FreeRTOS 操作系统实现多任务实时调度，保障端侧边缘 AI 算力与各控制模块高效协同。通过 ESP32-C3 模块连接微信小程序，基于 MQTT 协议将库存状态与出入库流水实时同步，实现多端闭环协同与 BOM 清单智能比对。系统集成 MIPI 图像采集与扫码模块，端侧部署两套 AI 模型分别进行人脸特征提取与精准识别，实现严密的离线权限管控，协同物料数字化录入，有效避免物资丢失与核对低效；集成 VL53L1 传感器进行距离检测，实现睡眠与工作模式的智能切换。结合立体机械防呆与图形界面交互，系统可精准驱动空间寻址（阵列亮灯/抽屉弹出），并通过语音助理与热敏打印获得即时反馈。整机采用模块化架构与四路独立供电轨设计，并针对大功率感性器件实施 200V 浪涌抑制，软硬件解耦清晰，抗干扰能力强。整体方案兼顾高精度与安全性，满足研发实验室对高交互性与高可靠性的需求。

1.4 主要性能指标

项目	指标值	测试条件
人脸识别准确率	$\geq 80\%$	室内环境下多次扫脸测试
二维码解析时间	$\leq 2000\text{ms}$	标准元器件袋标签扫码解码测试
FreeRTOS 任务响应	$< 10\text{ms}$	高密度状态机与多任务并发样机测试
MQTT 通信同步延迟	$\leq 70\text{ms}$	样机与微信小程序端数据交互测试
温湿度检测精度	$\pm 3^{\circ}\text{C}$	与室内温度计对比测得

1.5 主要创新点

- [1] 基于 STM32N647 与 FreeRTOS 多任务调度的端侧边缘 AI 智能仓储管理系统；
- [2] 端侧部署两套 AI 模型，实现本地离线的高精度人脸特征提取与安全权限管控；
- [3] 联动微信小程序 BOM 一键比对与 MQTT 协议，构建云边端多端协同数字化物料闭环；

[4] 结合阵列指示灯、推拉电磁铁与立体机械防呆结构，达成秒级高精寻址与物理防错机制；

[5] 设计电源路径管理电路，构建兼顾多场景适配的平滑切换的高可靠四路供电架构。

1.6 设计流程

设计过程中我们首先明确研发实验室物料管理的痛点与需求，建立整体功能框架；随后进行模块划分，包括主控、双 AI 模型、无线通信、执行机构、电源管理及多模态交互等；依照模块独立开发的顺序开展四路独立供电硬件设计与 PCB 原型搭建；并行进行 FreeRTOS 多任务调度与各子系统驱动开发；完成整机联调、本地 AI 调试与小程序 MQTT 对接；最后进行电气可靠性优化与低功耗平滑切换改进，形成完整的智能元器件仓储系统方案。

第二部分 系统组成及功能说明

2.1 整体介绍

本系统由软件与硬件两部分协同构成，各模块功能明确、互联互通，实现了从底层感知、端侧边缘 AI 处理到云端交互、物理执行的闭环控制。

软件部分：最底层是单片机的硬件资源：GPIO、I2C、SPI、USART、MIPI-CSI、DCMI 等。

在此之上是 module 层——使用了单片机这些硬件资源的同时组成了系统独立功能的芯片器件与算法组件：MIPI 图像采集驱动、双 AI 模型（人脸特征提取与精准识别模型）、高速二维码解码算法、VL53L1 激光测距 TOF 传感器、温湿度传感器、SK9822 阵列灯珠、语音助理以及热敏打印驱动。这一层承担着硬件操作与业务代码解耦的责任。

在 module 层之上是中间件层，它是由 FreeRTOS 实时操作系统以及基于 MQTT 协议的无线通信网络栈组成的。调用中间件层可以迅速搭建业务代码，加快开发进度。

软件架构最顶层是 app 层，它负责实现业务功能的部分，通过调用 RTOS 的 api 实现人脸权限管控、BOM 清单智能比对、物料空间寻址、动态库存预警、低功耗平滑切换等各个任务间的高效沟通以及单片机资源的合理利用，调用无线

网络快速实现与微信小程序的协同开发。这些让我们任务紧密合作而又互不干扰，为后续功能的拓展及优化奠定了基础。

硬件部分：它是围绕着 STM32N647 单片机电路为核心，以各个传感器及控制芯片外围电路为基本单元组成的一个合理有效的板上系统。

四路独立供电轨及驱动防护电路设计是硬件中极具挑战的部分。我们对电源拓扑进行了充分的思考，实现了端侧大算力 AI 瞬态大电流、无线射频功耗与电磁执行机构感性负载的物理隔离，并设计了完善的过压与反向电动势抑制回路。高速电路部分同样富有挑战，团队对 MIPI 差分信号进行了严格的阻抗匹配与信号完整性设计，在识别多个元器件标签与扫码时确保图像数据的稳定传输，保证能源充足利用与整机的高可靠性

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

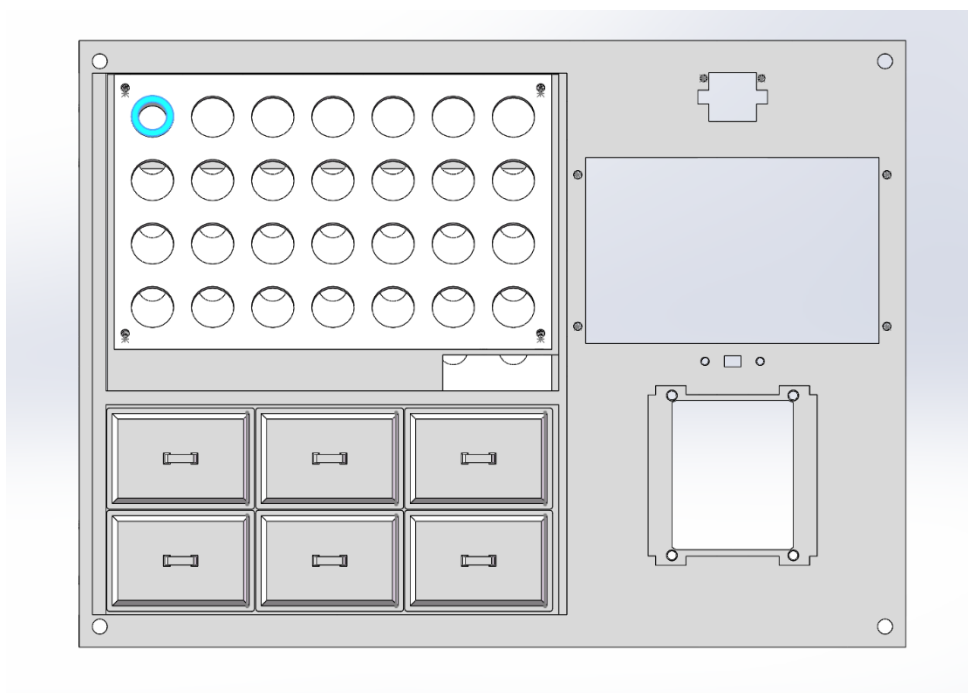
本系统硬件部分主要由机械结构与电路系统两大部分构成。机械结构包括大元件抽屉仓、小元件试管存储架、机械防呆结构与推拉电磁铁执行机构等，支持元器件的分区存储、抽屉自动弹开与柜门权限锁控，具备良好的物理防错与空间寻址指引设计，便于日常控制与多用户维护。

电路系统由核心主控底板（搭载正点原子 STM32N647 核心板）、摄像头图像采集板、SK9822 阵列指示灯板与热敏打印控制模块组成。核心主控部分依托正点原子核心板的高性能硬件算力，并配合自研底板上的四路独立供电轨防护电路，共同负责整个系统的边缘 AI 计算、电源平滑切换与信号调度。各功能模块之间通过高速总线或标准串行通信接口连接，具备优异的抗干扰性能与明确的解耦设计，便于系统装配与工程调试

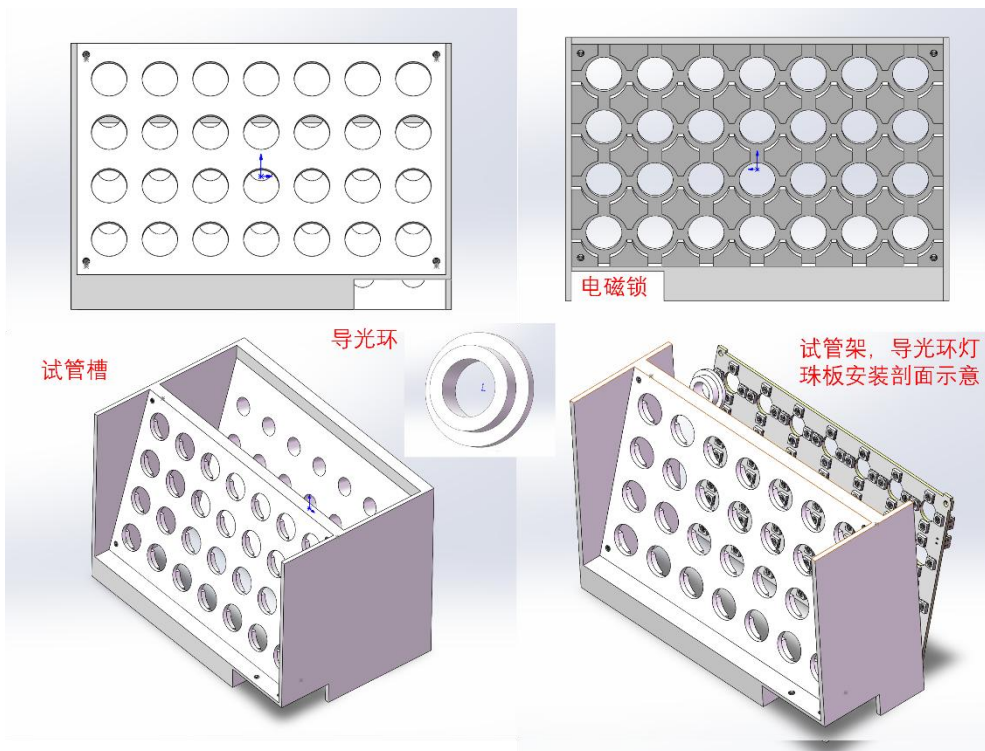
2.2.2 机械设计介绍

本系统的机械结构设计遵循从“总体框架”到“局部组件”的逐级递进原则。考虑到若整体柜身全部采用 3D 打印将会导致制作成本极其高昂，团队在结构方案上进行了优化设计：首先选用 2020 标准铝型材搭建出整体的主体框架，从根本上保证了柜体核心的机械强度与结构刚度；在此基础上，系统外

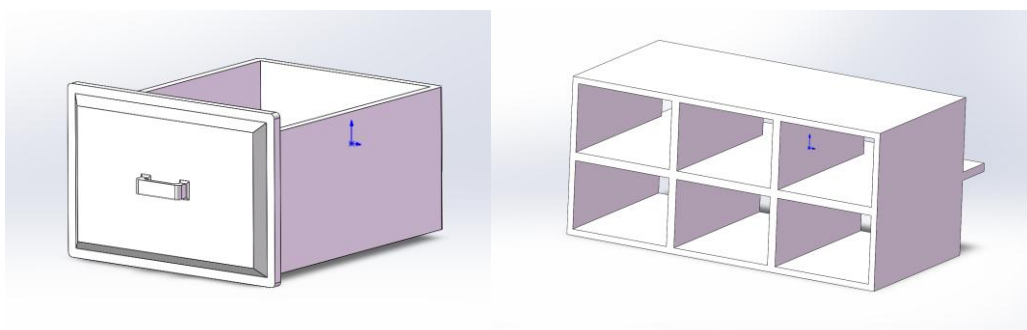
侧的非精密包裹面均采用白色或透明亚克力板进行数字化切割定制，既降低了整机重量与加工成本，又便于观察内部结构；而系统正面的正面板、试管架、导光环、小抽屉以及抽屉架等高精度配合与复杂曲面结构，则全部采用高性能 3D 打印工艺进行精密加工，实现了结构刚性、制造成本与装配精度的完美平衡。



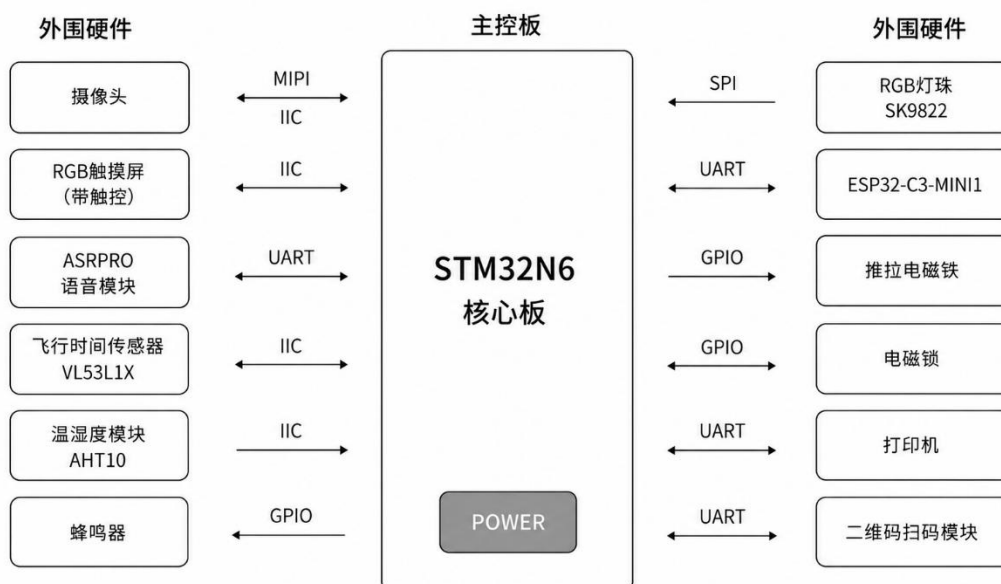
在局部组件设计中，首先是小元件试管存储架部分，如图 2-3 所示。该组件完全基于 3D 打印实现定制化造型，每个微型试管存储位均预留了专用的导光环安装卡槽，用于固装高密度导光环。为了解决日常使用中微型试管因震动或误触容易向外滑落丢失的工程痛点，团队在机械结构上进行了人性化的防摔防呆设计，将试管架的放置面精心设计为向内倾斜 15° 的特定夹角。这一重力自锁机制能够确保试管在无外力干预下牢固稳居于槽位内，有效提升了物料存取的物理安全性。



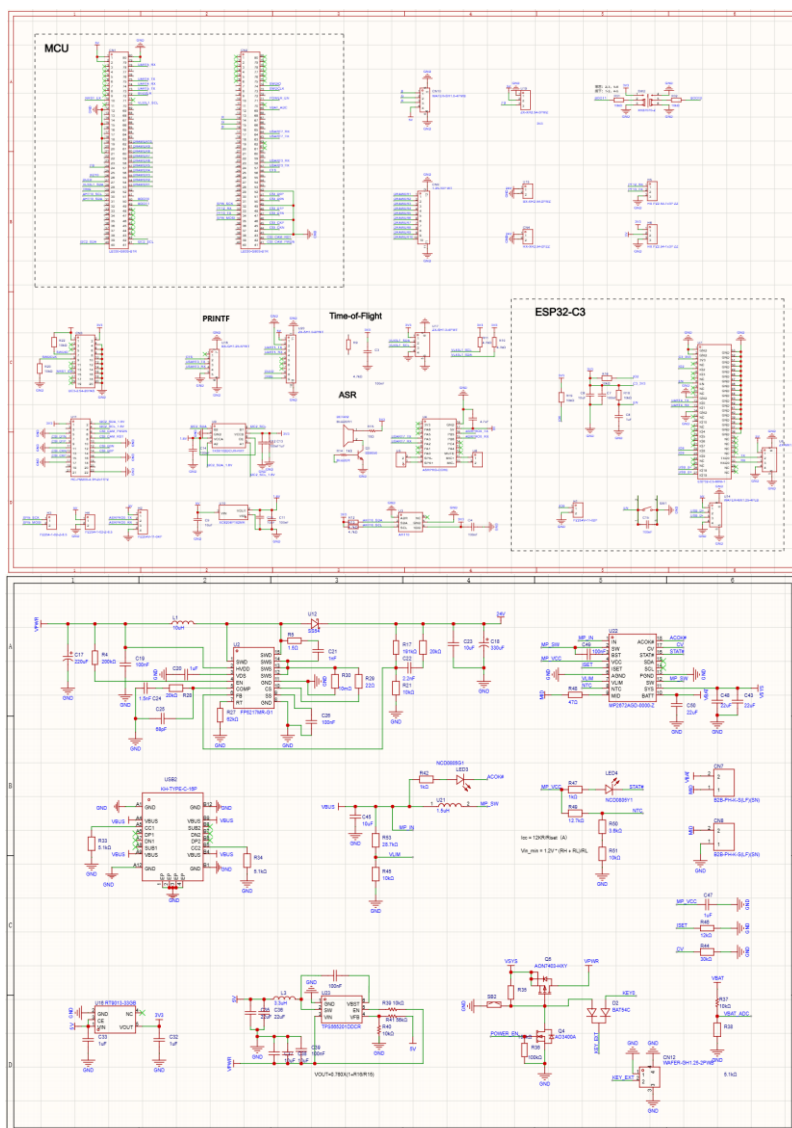
其次是大元件抽屉仓与执行机构部分，如图 2-4 所示。由于小抽屉与抽屉架之间存在高频的推拉摩擦与定位配合要求，该部分采用了模块化 3D 打印框架设计。抽屉架内部精密集成了推拉式电磁铁的固定支架与复位触发组件，当主控板释放解锁信号驱动电磁铁动作时，推杆能够精准作用于小抽屉后端，驱动抽屉完成秒级自动弹开。整个小抽屉的滑道阻尼、限位防脱机构以及与电磁锁销的机械咬合面均经过高精度建模微调，配合立体的防呆卡位，确保了多通道抽屉在频繁弹出与闭合过程中的顺畅性与长效可靠性。



2.2.3 电路各模块介绍



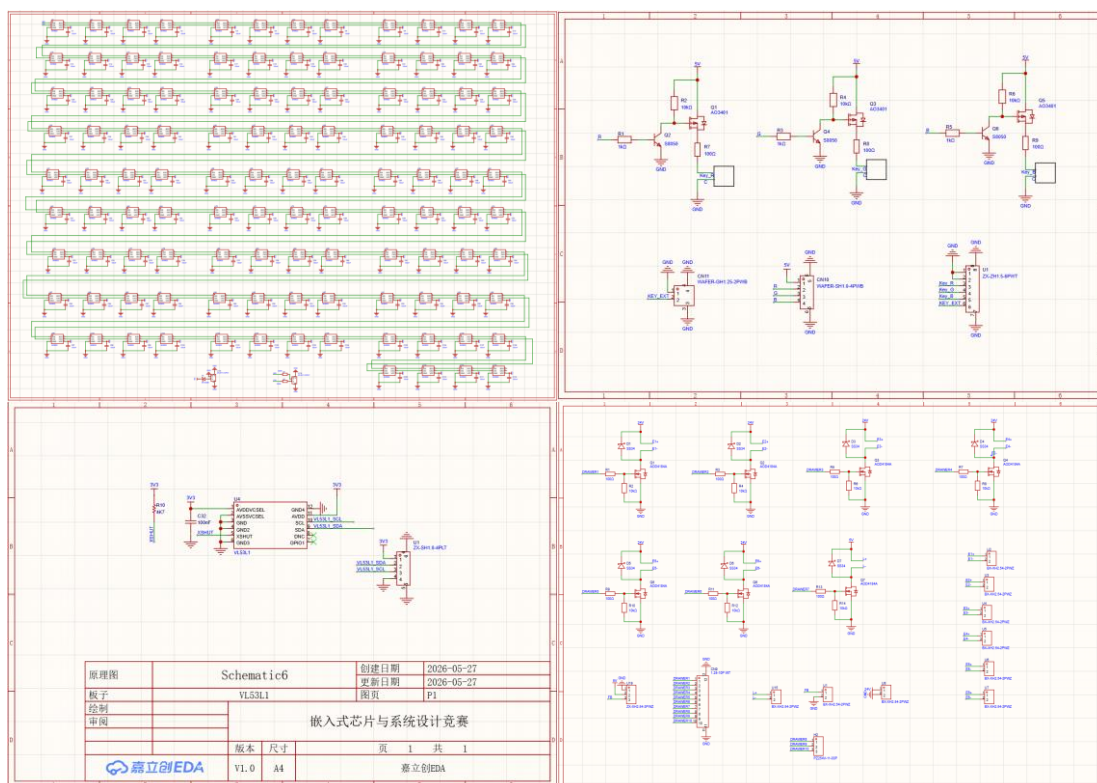
如图所示，整套电路系统由核心主控底板（搭载正点原子核心板）、摄像头图像采集板、SK9822 阵列指示灯板与热敏打印控制模块等多张功能电路板组成。其中主控底板是核心模块，集成了 STM32N6 核心主控芯片，并集成了多种传感与交互功能模块，包括：摄像头、RGB 触摸屏（带触控）、ASRPRO 语音模块、飞行时间传感器 VL53L1X、温湿度模块 AHT10 与蜂鸣器等。主控板还集成了电源与系统支持模块，如双节锂电池全链路电源管理电路、满足大功率执行器与低功耗主控所需的 24V/5V/3.3V/1.8V 四路独立供电轨、电磁隔离驱动与防护电路及智能低功耗控制机制等，为整个系统稳定运行提供保障。其余功能电路板通过板上的 FPC 或 XH2.54 接口进行有线通信，且采用可拆卸结构设计，便于系统装配与后期维护



上面两张图展示了本系统的主控板原理图本系统的核心底板电路主要由电源管理拓扑、外设通信接口以及电磁执行机构驱动电路三大部分组成。整个电路设计紧密围绕正点原子 STM32N647 核心板展开，通过合理的板级电路布局，实现了全系统高效的电源分配与多外设协同控制。

在外设接口与通信互联方面，底板提供了丰富且规范的引脚引出与外设接口。其中，ESP32-C3 无线通信模块与 ASRPRO 语音助理模块通过独立的 UART 串口与 STM32N647 进行高速数据异步通信，实现云端 MQTT 对接与本地语音控制；温湿度传感器 AHT10 与激光测距传感器 VL53L1X 共用一条硬件 I2C 总线进行环境数据采集与防呆接近检测；并且预留调试串口方便整机固件烧录与日志查看。

所有外设接口均布设了阻容滤波与上拉电阻，全面保证了信号在多板卡传输过程中的完整性与抗干扰能力。



在整套硬件系统中，除核心主控底板外，系统还设计并集成了多个独立的数字化扩展功能板与专项驱动电路，以完成高精度的物料寻址指示、空间防呆测距、人机交互输入以及大功率机构物理执行。各部分电路方案具体设计如下：

第一，SK9822 阵列指示灯板电路设计。该灯板主要用于试管存储架的立体空间寻址指引。电路核心采用双线式双向控制的高精度 SK9822 型三色 RGB 灯珠，通过串行级联方式构成高密度指示灯阵列。相较于传统单线制灯珠，该电路引入了独立的时钟线与数据线，通过主控芯片的同步串行接口或类 SPI 总线进行高速数据传输，具备极高的刷新率与抗干扰性能。每个灯珠引脚处均就近并联了去耦滤波电容，能够有效抑制高频色彩切换时产生的瞬态电流噪声，确保灯带亮度均匀且数据传输稳定。

第二，激光测距模块电路设计。该模块主要负责试管架区域的防呆接近检测与存取状态感知。电路基于飞行时间技术，集成高性能红外激光测距传感器。该模块通过标准 I2C 总线与主控底板进行双向异步通信，板级电路上布设了专用的 I2C 上拉电阻以及多级低阻抗滤波电容，以保证通信信号的完整性。

第三，按键输入驱动电路设计。为了满足本地的手动控制与应急功能切换，

系统设计了独立的机械式按键外设电路。该电路在常规按键开关系列的基础上，加入了完善的硬件去抖动网络。按键信号线通过限流电阻连接至主控 GPIO 引脚，并在引脚端对地并联了旁路滤波电容，利用 RC 低通滤波器的充放电特性，从硬件源头上彻底滤除机械按键按下与释放瞬间产生的接触毛刺与弹性抖动噪声，显著减轻了主控芯片软件轮询或中断解码的计算开销。

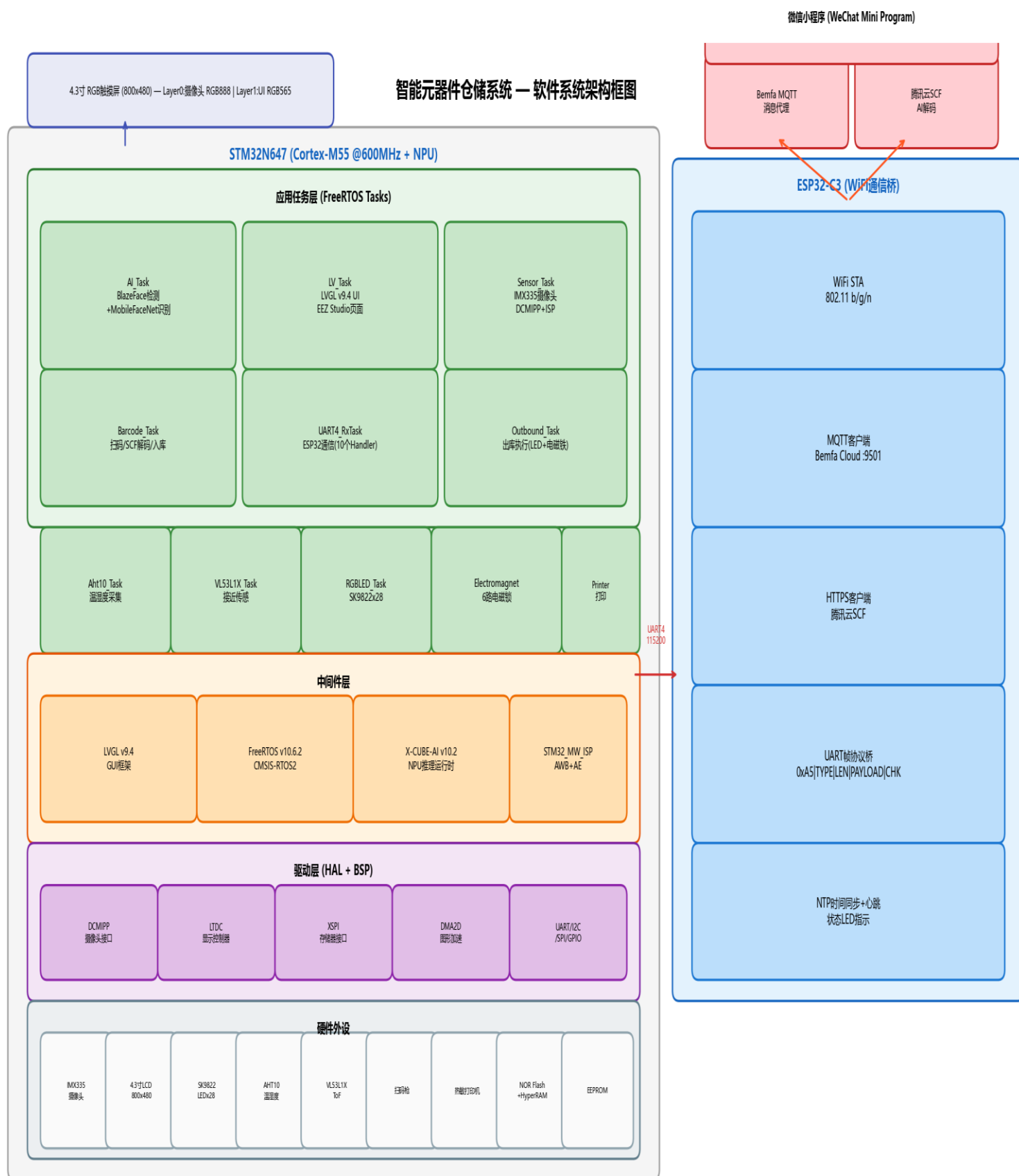
第四，电磁铁执行机构驱动电路设计。该电路是驱动大元件仓抽屉自动弹开与柜门锁控的关键功率单元。由于推拉电磁铁属于典型的大功率强感性负载，为防止其动作瞬间产生的高频电气噪声干扰主控弱电系统，电路采用了完善的光耦隔离驱动架构。主控芯片输出的逻辑电平仅用于控制光电耦合器内部发光二极管的导通，从物理层面上将弱电控制级与 24V 强电驱动级完全隔离开来。在功率输出端，选用低导通内阻、大电流的 N 沟道增强型功率场效应管作为开关控制核心，并在电磁铁接线端子两端反向并联了高速大电流续流二极管。当场效应管关断瞬间，续流二极管能够迅速释放感性线圈产生的反向电动势与高压瞬态尖峰，从而保护驱动管不被击穿，保障了整机系统的电气可靠性

2.3 软件系统介绍

2.3.1 软件整体介绍（含 PC 端或云端，结合关键图片）；

2.3.1.1 系统整体框图

系统整体软件架构如下图所示，从顶层到底层划分为应用任务层、中间件层、驱动层和硬件外设层，右侧为 ESP32-C3 通信桥模块和云端服务。

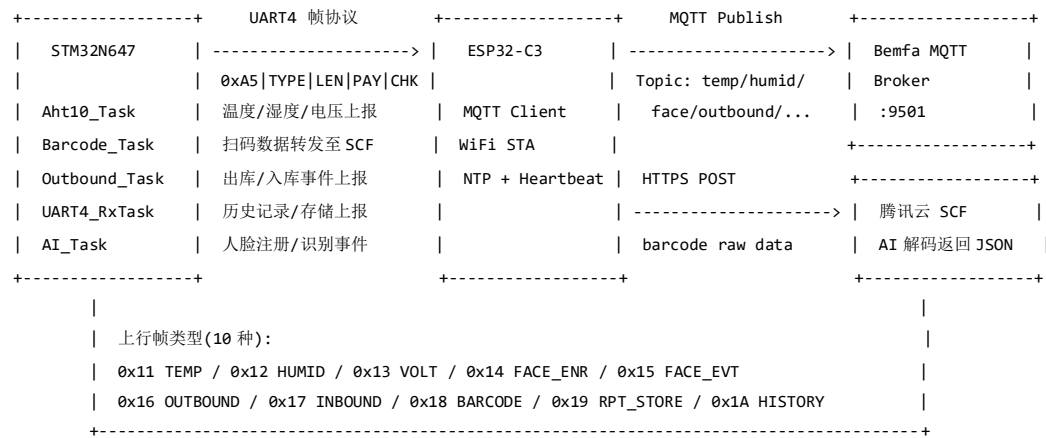


2.3.1.2 软件技术栈

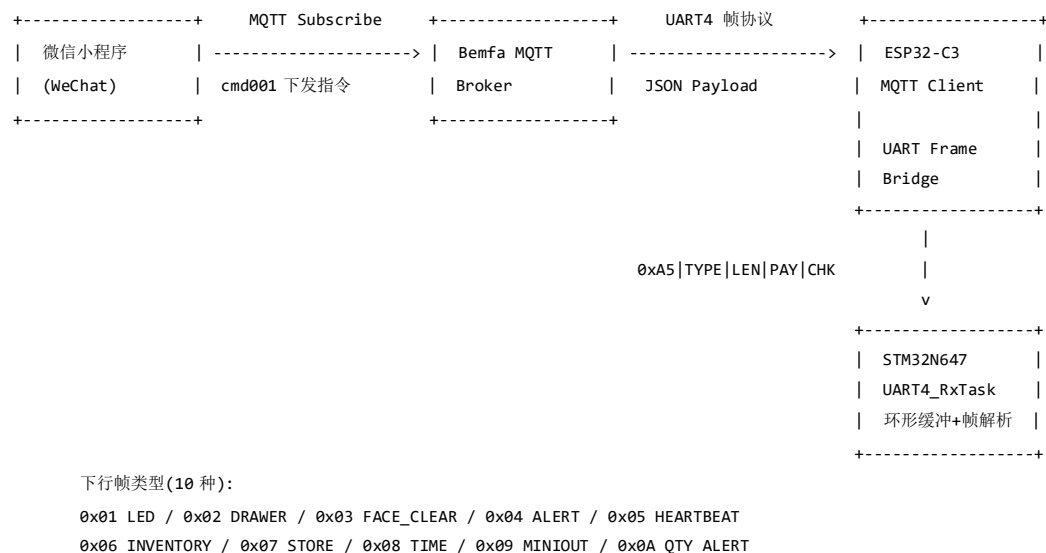
层次	技术/组件	版本/型号	说明
MCU 主控	STM32N647	Cortex-M55 @600MHz	内 置 NPU(MAC:250 GOPS, 300MHz)
通信协处理	ESP32-C3	RISC-V @160MHz	WiFi 4 + BLE 5.0
RTOS	FreeRTOS	v10.6.2 (CMSIS-RTOS2)	13 个任务，抢占式调度
AI 框架	X-CUBE-AI	v10.2.0	NPU 推理运行时，双模型 管线
人脸检测	BlazeFace	128x128x3 输入	101K 参数，NPU 推理，6 关键点
人脸识别	MobileFaceNet	112x112x3 输入	993K 参数，NPU+CPU 混 合推理
UI 框架	LVGL	v9.4	EEZ Studio 可视化设计， 4 个页面
显示驱动	LTDC	双图层	Layer0:RGB888(摄像头), Layer1:RGB565(UI)
摄像头接口	DCMIPP	双管道	PIPE1:预览(PxCA 叠加), PIPE2:AI 输入
摄像头传感器	IMX335	5MP	ISP 内置 AWB/AE
存储器	NOR Flash + HyperRAM	XSPI 接口	程序存储+帧缓冲+AI 权 重
LED 驱动	SK9822	SPI5	28 颗 RGB LED, 单线控制
电磁锁	GPIO 直驱	6 路(PG14/PB13/PQ4-7)	500ms 脉冲开锁
温湿度传感器	AHT10	I2C(软件模拟)	精度: +/-0.3C, +/-2%RH
ToF 测距	VL53L1X	I2C2	最远 4m, 用于屏幕休眠 控制
扫码枪	USB 串口	UART5 @9600	接收条码原始数据
热敏打印机	TTL 串口	UART7	GBK 编码, BOM 标签打 印
STM32-ESP32 通信	UART4	115200bps	自定义二进制帧协议
MQTT 协议	Bemfa Cloud	bemfa.com:9501	10 个上行 Topic + 1 个下 行 Topic
云 AI 解码	腾讯云 SCF	HTTPS POST	条形码原始数据上传→AI 识别→JSON 返回
移动端	微信小程序	WeChat Mini Program	库存查询/出库操作/预警 推送

2.3.1.3 双 MCU 数据流

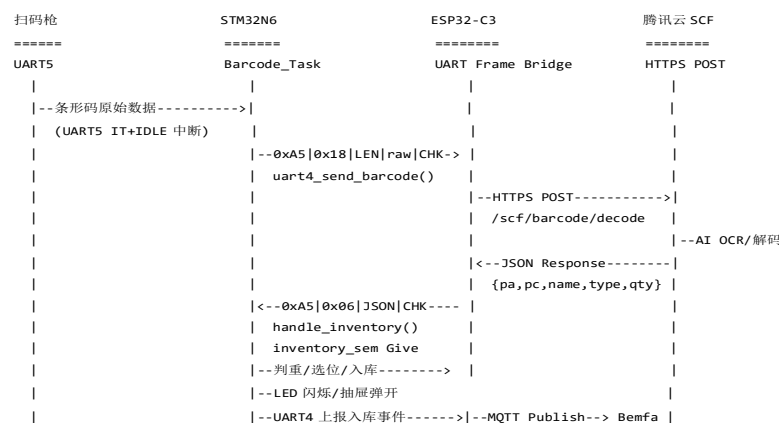
2.3.1.3.1 上行数据流 (STM32 -> ESP32 -> Cloud)



2.3.1.3.2 下行数据流 (Cloud -> ESP32 -> STM32)



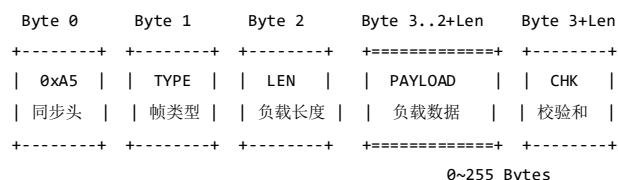
2.3.1.4 扫码入库数据流



2.3.1.5 UART 帧协议设计

STM32N647 与 ESP32-C3 之间通过 UART4(115200bps)进行数据交换，采用自定义二进制帧协议，帧结构紧凑高效，适用于嵌入式串行通信场景。

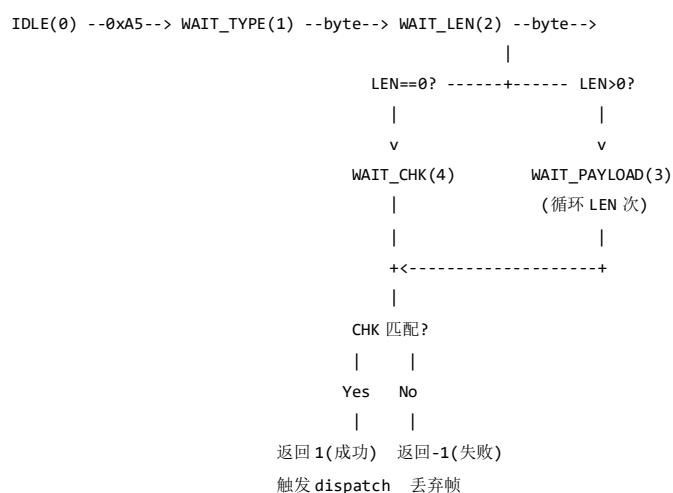
2.3.1.5.1 帧格式



CHK = TYPE xor LEN xor PAYLOAD[0] xor ... xor PAYLOAD[LEN-1]

帧总长度 = 4 + LEN (最小 4 字节, 最大 259 字节)

2.3.1.5.2 帧解析状态机



2.3.1.5.3 帧类型总表

方向	TYPE	宏定义	Payload 格式	用途
ESP32->STM32	0x01	FRAME_TYPE_LED	bitmap[4B] MSB	控制 28 颗 LED 闪烁
ESP32->STM32	0x02	FRAME_TYPE_DRAWER	mask[1B]	控制 6 路电磁锁弹开
ESP32->STM32	0x03	FRAME_TYPE_FACE_CLEAR	(空)	清除人脸库
ESP32->STM32	0x04	FRAME_TYPE_ALERT	sub[1B]+val[2B]BE	温湿度阈值告警
ESP32->STM32	0x05	FRAME_TYPE_HEARTBEAT	status[1B]	心跳+状态同步
ESP32->STM32	0x06	FRAME_TYPE_INVENTORY	pa\0pc\0name\0type\0qty[2B]src[1B]	SCF 解码结果回传
ESP32->STM32	0x07	FRAME_TYPE_STORE	type[1B]+slot[1B]+flag[1B]	小程序入库指令
ESP32->STM32	0x08	FRAME_TYPE_TIME	year[2B]mon[1B]day[1B]h[1B]m[1B]s[1B]	NTP 时间同步
ESP32->STM32	0x09	FRAME_TYPE_MINIOUT	type[1B]+slot[1B]+qty[2B]BE	小程序出库指令
ESP32->STM32	0x0A	FRAME_TYPE_QTY_ALERT	pc\0cab[1B]qty[2B]BE	库存预警通知
STM32->ESP32	0x11	FRAME_TYPE_TEMP	raw[2B]BE (x10)	温度数据上报
STM32->ESP32	0x12	FRAME_TYPE_HUMID	raw[2B]BE (x10)	湿度数据上报
STM32->ESP32	0x13	FRAME_TYPE_VOLT	raw[2B]BE (x10)	电压数据上报

STM32->ESP32	0x14	FRAME_TYPE_FACE_ENR	id_str[N]	人脸注册事件
STM32->ESP32	0x15	FRAME_TYPE_FACE_EVT	event_str[N]	人脸识别事件
STM32->ESP32	0x16	FRAME_TYPE_OUTBOUND	info_str[N]	出库结果上报
STM32->ESP32	0x17	FRAME_TYPE_INBOUND	info_str[N]	入库结果上报
STM32->ESP32	0x18	FRAME_TYPE_BARCODE	raw_barcode[N]	扫码原始数据转发
STM32->ESP32	0x19	FRAME_TYPE_RPT_STORE	location[N]	存储位置上报
STM32->ESP32	0x1A	FRAME_TYPE_HISTORY	time\0action\0pc\0user\0cab[1B]qty[2B]	历史记录上报

2.3.1.6 FreeRTOS 任务架构

系统共创建 13 个 FreeRTOS 任务，采用抢占式调度，基于 CMSIS-RTOS v2 API 实现。任务间通过二值信号量 (cam_frame_sem, inventory_sem) 和互斥锁 (i2c2_mutex, flow_var_mutex) 进行同步。

任务名	入口函数	优先级	栈大小	核心职责
defaultTask	StartDefaultTask	Normal(8)	2KB	系统空闲任务，心跳 LED 闪烁
SensorTask	Sensor_Task	Normal+1(9)	2KB	IMX335 摄像头初始化+DCMIPP 连续采集+ISP 后台
AITask	AI_Task	Normal+1(9)	8KB	BlazeFace 检测+MobileFaceNet 识别双模型管线
LV_Task	LVGL_Task	Normal(8)	96KB	LVGL v9.4 渲染+LTDC Layer1 UI+EEZ Flow tick
BarcodeTask	Barcode_Task	Normal(8)	8KB	UART5 扫码接收→SCF 解码等待→入库处理
UART4_RxTask	UART4_RxTask	Normal(8)	2KB	环形缓冲接收→帧解析状态机→10 个 Handler 分发
OutboundTask	Outbound_Task	Normal(8)	2KB	遍历 judgment_out 标记→聚合 LED 位图+抽屉掩码
RGBLEDTask	RGBLED_Task	Normal(8)	1KB	28 位位图→SK9822 红色闪烁(6 秒)
ElectromagnetTask	Electromagnet_Task	Normal(8)	2KB	6 路 GPIO 500ms 脉冲控制电磁锁
Aht10Task	Aht10_Task	Low(4)	2KB	AHT10 温湿度 2s 周期采集+上报
VL53L1XTask	VL53L1X_Task	Normal(8)	1KB	ToF 距离检测→屏幕背光自动控制
PrinterTask	Printer_Task	Normal+1(9)	2KB	热敏打印机 BOM 标签输出
ElectromagnetLockTask	ElectromagnetLock_Task	Normal(8)	0.5KB	电磁锁关闭脉冲控制

2.3.1.7 MQTT 主题映射

系统使用 Bemfa Cloud MQTT Broker(bemfa.com:9501)作为消息中间件，STM32 通过 ESP32-C3 间接发布/订阅 MQTT 消息，实现与微信小程序的双向通信。

方向	Topic	数据内容	QoS
上行(发布)	temp	温度值(摄氏度, float)	0
上行(发布)	humid	湿度值(百分比, float)	0
上行(发布)	volt	电压值(伏特, float)	0
上行(发布)	face_enr	人脸注册事件(用户名)	0
上行(发布)	face_evt	人脸识别事件(用户名)	0
上行(发布)	outbound	出库事件(位置+数量)	0
上行(发布)	inbound	入库事件(位置+数量)	0
上行(发布)	store	存储位置信息	0
上行(发布)	history	历史记录(JSON)	0
上行(发布)	bar_result	SCF 解码结果透传	0
下行(订阅)	cmd001	微信小程序控制指令 (JSON)	1

2.3.2 软件各模块介绍（根据总体框图，给出各模块的具体设计说明。从顶层到底层逐次给出各函数的流程图及其关键输入、输出变量）；

2.3.2.1 系统初始化流程 (main.c + app_freertos.c)

系统上电后，main.c 依次完成 HAL 库初始化、外设时钟配置、XSPI 存储器初始化(NOR Flash+HyperRAM)、LCD 初始化、串口初始化、X-CUBE-AI NPU 运行时初始化。随后调用 osKernelInitialize()初始化 FreeRTOS 内核，MX_FREERTOS_Init()创建信号量/互斥锁/任务，最后 osKernelStart()启动调度器。

2.3.2.1.1 关键函数

函数名	功能描述	关键参数
main()	系统入口函数，依次初始化所有外设并启动 FreeRTOS	void
MX_FREERTOS_Init()	创建互斥锁、信号量、13 个 FreeRTOS 任务	void
SystemIsolation_Config()	配置 RIF 安全隔离，设置 DCMIPP/DMA2D/LTDC/NPU 主设备属性	void
StartDefaultTask()	系统空闲任务,周期性翻转 PG10 作为心跳指示	void *argument

2.3.2.1.2 流程图

```

main()
|
+--> SCB->VTOR 设置 (Debug:0x34000400 / Release:0x70100400)
+--> HAL_Init() + SystemCoreClockUpdate()
+--> MX_GPIO/DCMIPP/DMA2D/I2C/LTDC/UART/XSPI/SPI/TIM Init
+--> MX_X_CUBE_AI_Init() - NPU 运行时初始化
+--> SystemIsolation_Config() - RIF 安全隔离
+--> XSPI1 HyperRAM Init + MemoryMapped
+--> XSPI2 NOR Flash Init + MemoryMapped
+--> rgblcd_init() + rgblcd_display_dir(1)
+--> uart_init(115200)
+--> osKernelInitialize() - FreeRTOS 内核初始化
+--> MX_FREERTOS_Init()
|   +--> osMutexNew(i2c2_mutex, flow_var_mutex)
|   +--> osSemaphoreNew(cam_frame_sem, inventory_sem)
|   +--> osThreadNew: Sensor/LV/Aht10/UART4_Rx/VL53L1X
+--> osKernelStart() - 启动调度器
|
(调度器接管，各任务按优先级抢占执行)

```

2.3.2.1.3 关键输入/输出变量

变量名	类型	方向	说明
SystemCoreClock	uint32_t	全局	系统核心时钟频率 (600MHz)
cam_frame_sem	osSemaphoreId_t	全局	摄像头帧就绪信号量 (AI_Task 等待)
inventory_sem	osSemaphoreId_t	全局	SCF 解码响应信号量 (Barcode_Task 等待)
i2c2_mutex	osMutexId_t	全局	I2C2 总线互斥锁(触摸屏 + VL53L1X 共享)
flow_var_mutex	osMutexId_t	全局	EEZ Flow 全局变量互斥锁

2.3.2.2 AI 人脸识别模块 (AI_Task)

AI_Task 是整个系统的核心智能模块，运行双模型 AI 管线：BlazeFace 进行人脸检测 (128x128x3 输入，NPU 推理)，MobileFaceNet 进行人脸特征提取与识别(112x112x3 输入，NPU+CPU 混合推理)。模块通过 DCMIPP PIPE2 获取 AI 专用图像帧，通过 cam_frame_sem 信号量同步帧到达事件。支持人脸注册(multi-enroll)、人脸识别(identify)和人脸库查询(gallery)三种 ReID 模式。

2.3.2.2.1 关键函数

函数名	功能描述	关键参数
AI_Task()	主循环：等待信号量->uint8 转 float32->NPU 推理 BlazeFace->检测框绘制->串口命令处理->ReID 管线	void *argument
MX_X_CUBE_AI_Process_User()	X-CUBE-AI 生成的推理封装函数，执行 BlazeFace NPU 推理+后处理	ai_result_t *result
process_serial_commands()	解析 UART1 串口命令：register/s/identify/gallery/help	void
run_reid_pipeline()	ReID 管线：人脸裁剪 112x112->量化 int8->MobileFaceNet 推理->注册或识别	ai_result_t *result
draw_detections_on_display()	在 LTDC Layer0 帧缓冲上绘制检测框 (暂停 PIPE1 防止覆盖)	ai_result_t *result
draw_one_detection()	用 DMA2D R2M 模式绘制单个检测框(绿色矩形)	ai_detection_t *d, r, g, b

2.3.2.2.2 流程图

AI_Task 主循环

```

|
+--> osSemaphoreAcquire(cam_frame_sem, INFINITE) - 等待摄像头帧
+--> SCB_CleanInvalidatedDCache(nn_input_u8) - 缓存一致性维护
+--> uint8->float32 转换: nn_input_u8[i]/255.0f -> buffer_in
+--> SCB_CleanInvalidatedDCache(buffer_in)
+--> MX_X_CUBE_AI_Process_User(&result) - BlazeFace NPU 推理
|
|   +--> network_f Run (NPU)
|   +--> 后处理: NMS + 检测框解析 + 关键点提取
|   +--> 填充 ai_result_t (nb_detect, detections[])
+--> draw_detections_on_display() - 绘制绿色检测框
+--> process_serial_commands() - 串口命令处理
|
|   +--> 'register <name>': 启动多角度注册, g_capture_pending=1
|   +--> 's': 采集下一个角度样本
|   +--> 'identify': 触发单次识别
|   +--> 'gallery': 打印已注册人脸列表
+--> run_reid_pipeline()
|
|   +--> ai_crop_resize_face_112() - 从显示帧裁剪人脸区域
|   +--> quantize_u8_to_s8() - uint8->int8 量化
|   +--> SCB_CleanInvalidatedDCache(buffer_in_fc)
|   +--> ai_face_reid_run() - MobileFaceNet 推理 -> embedding[128]
|   +--> REID_REGISTER: ai_face_enroll_multi() - 多角度注册
|   +--> REID_IDENTIFY: ai_face_identify() - 余弦相似度匹配
|       +--> g_identify_match=1, g_identify_done=1
+--> (UI 触发识别)g_trigger_identify -> reid_pending=REID_IDENTIFY
+--> vTaskDelay(1ms) -> 下一帧循环

```

2.3.2.2.3 关键输入/输出变量

变量名	类型	方向	说明
nn_input_u8[128*128*3]	uint8_t[]	输入	DCMIPP PIPE2 DMA 写入的 AI 输入帧(RGB888)
buffer_in	float32[]	中间	nn_input_u8 归一化后的 float32 输入(BlazeFace)
ai_result_t	struct	输出	BlazeFace 推理结果(nb_detect, detections[])
g_identify_name[32]	char[]	输出	识别匹配到的用户名
g_identify_dist	float	输出	识别匹配的余弦距离
g_identify_match	volatile int	输出	识别匹配标志(1=成功, 0=未匹配)
buffer_in_fc	int8_t[]	中间	MobileFaceNet 输入(int8 量化后的人脸 112x112)
embedding[128]	float[]	中间	MobileFaceNet 输出的人脸特征向量

2.3.2.3 摄像头传感器模块 (Sensor_Task)

Sensor_Task 负责 IMX335 摄像头的初始化配置和 DCMIPP 连续采集控制。上电后通过软件 I2C 配置 IMX335 寄存器(白平衡模式 6500K)，配置完成后释放 I2C 引脚给触摸屏使用。启动 DCMIPP PIPE1(预览输出到 LTDC Layer0,RGB888)和 PIPE2(AI 输入,128x128 RGB888 到 nn_input_u8)，PIPE2 每完成一帧通过 HAL_DCMIPP_PIPE_FrameEventCallback 回调释放 cam_frame_sem 信号量通知 AI_Task。主循环中周期性调用 ISP 后台处理(AWB/AE)。

2.3.2.3.1 关键函数

函数名	功能描述	关键参数
Sensor_Task()	IMX335 初始化->白平衡->释放 I2C->启动连续采集->ISP 后台循环	void *argument
imx335_init()	通过软件 I2C 配置 IMX335 寄存器(分辨率/帧率/MIPI 等)	void
imx335_set_wb_mode()	设置白平衡模式(6500K 日光)	uint32_t mode
imx335_io_deinit()	释放 IMX335 占用的 I2C IO(PD14/PC2)，交还给触摸屏	void
imx335_start_capture()	启动 DCMIPP PIPE1+PIPE2 双管道连续采集	uint32_t fb_addr
imx335_isp_background_process()	ISP 后台处理(AWB/AE 自动调整)，每 10ms 调用一次	void
HAL_DCMIPP_PIPE_FrameEventCallback()	PIPE2 帧完成回调：释放 cam_frame_sem 通知 AI_Task	hdcmipp, Pipe

2.3.2.3.2 流程图

```

Sensor_Task
|
+--> HAL_GPIO_WritePin(PG4=1, PG6=1) - 摄像头电源使能
+--> imx335_init() - 软件 I2C 配置 IMX335 寄存器
|   +--> 循环重试直到成功 (while(imx335_init()) { __NOP(); })
+--> imx335_set_wb_mode(6500) - 白平衡设置
+--> imx335_io_deinit() - 释放 I2C IO(PD14, PC2)给触摸屏
+--> imx335_start_capture(g_ltdc_framebuf) - 启动双管道采集
|   +--> PIPE1: 连续模式 -> LTDC Layer0 (RGB888, 800x480)
|   +--> PIPE2: 连续模式 -> nn_input_u8 (RGB888, 128x128)
|       +--> 每帧完成后 HAL_DCMIPP_PIPE_FrameEventCallback
|       +--> osSemaphoreRelease(cam_frame_sem)
+--> 主循环: imx335_isp_background_process() + vTaskDelay(10ms)
  
```

2.3.2.3.3 关键输入/输出变量

变量名	类型	方向	说明
g_ltdc_framebuf[800*480*3]	uint8_t[]	输入	PIPE1 预览输出目标 (LTDC Layer0 帧缓冲)
nn_input_u8[128*128*3]	uint8_t[]	输出	PIPE2 AI 输出目标(DMA 写入.noncacheable 段)
cam_frame_sem	osSemaphoreId_t	输出	PIPE2 帧完成信号(释放给 AI_Task)
hdcmi	DCMI_HANDLETypeDef	全局	DCMI 外设句柄 (PIPE1/PIPE2 配置)

2.3.2.4 UI 渲染模块 (LV_Task)

LV_Task 是图形用户界面的核心渲染任务，基于 LVGL v9.4 框架和 EEZ Studio 可视化设计工具。模块配置 LTDC Layer1 作为 UI 图层(800x480)，叠加在 Layer0 摄像头层之上，通过 LTDC 硬件 Alpha 混合实现摄像头预览与 UI 控件的同屏显示。使用 DMA2D 进行 LVGL 部分缓冲的硬件加速 flush。UI 包含 4 个页面:锁屏页(Lock)、摄像头预览页(Camera)、主仪表盘(Dashboard)、选位页(Selection)。

2.3.2.4.1 关键函数

函数名	功能描述	关键参数
LVGL_Task()	延时 3s 等待外设 ->lv_port_init()->ui_init()->主循环 lv_timer_handler+ui_tick	void *argument
lv_port_init()	LVGL 初始化+LTDC Layer1 配置+部分缓冲+DMA2D flush 注册+触摸初始化	void
ui_init()	EEZ Flow UI 初始化(创建 4 个页面+控件+全局变量绑定)	void
ui_tick()	EEZ Flow tick 驱动(动画/定时器/事件处理)	void

2.3.2.4.2 流程图

```

LVGL_Task
|
+--> vTaskDelay(3000ms) - 等待 Sensor_Task/LCD 初始化完成
+--> lv_port_init()
|   +--> lv_init() - LVGL 内核初始化
|   +--> HAL_LTDC_ConfigLayer(Layer1, RGB565, 800x480)
|   +--> LVGL 部分缓冲分配(buf1/buf2, 各 1/10 屏幕)
|   +--> lv_st_ltdc_create_partial() - DMA2D flush 回调注册
|   +--> lv_port_indev_init() - 触摸屏 GT9xxx 输入初始化
|   +--> lv_tick_set_cb(xTaskGetTickCount) - 时基设置
+--> ui_init() - EEZ Flow 页面初始化
|   +--> 页面 1: Lock Screen (锁屏)
|   +--> 页面 2: Camera Preview (摄像头实时预览 Layer0)
|   +--> 页面 3: Dashboard (温湿度/库存/历史/预警)
|   +--> 页面 4: Selection Dialog (选位对话框)
+--> g_lv_stack_high_water = uxTaskGetStackHighWaterMark(NULL)
+--> 主循环 (5ms 周期):
|   +--> lv_timer_handler() - LVGL 定时器+渲染调度
|   +--> ui_tick() - EEZ Flow tick 驱动

```

2.3.2.4.3 关键输入/输出变量

变量名	类型	方向	说明
g_ltdc_ui_framebuf[800*480*2]	uint8_t[]	输出	LTDC Layer1 RGB565 帧缓冲(EXTRAM 段)
g_lv_stack_high_water	UBaseType_t	输出	LV_Task 剩余栈空间 (用于内存诊断)
hltdc	LTDC_HandleTypeDef	全局	LTDC 外设句柄(双图层配置)

2.3.2.5 扫码入库模块 (Barcode_Task)

Barcode_Task 负责接收扫码枪通过 UART5 发送的条形码原始数据，将数据通过 UART4 帧协议转发至 ESP32-C3，由 ESP32-C3 通过 HTTPS 请求腾讯云 SCF 进行 AI 解码识别。模块使用 inventory_sem 信号量等待 SCF 解码结果返回(超时 15 秒)，收到 InventoryItem 后执行判重逻辑:如果已存在于 28 个 T 位或 6 个 D 位中则累加数量，否则弹出选位对话框供用户选择存储位置。最终触发 LED 闪烁或电磁锁弹开作为物理指示。

2.3.2.5.1 关键函数

函数名	功能描述	关键参数
Barcode_Task()	主循环:等待 UART5 IDLE 中断->发送条码->等信号量->判重入库/LED 抽屈	void *argument
UART5_RxCallback()	UART5 RXNE 中断回调:逐字节存入环形缓冲	uint8_t byte
UART5_IDLE_Callback()	UART5 IDLE 中断回调:置位 uart5_data_ready 通知任务处理	void
uart4_send_barcode()	构建 FRAME_TYPE_BARCODE 帧发送至 ESP32-C3	data, len
inventory_store_to_slot()	将 InventoryItem 存入指定槽位(T00-T27 或 D00-D05)	item, location
inventory_add_quantity()	在已有槽位上累加库存数量	item, location

2.3.2.5.2 流程图

```

Barcode_Task
|
+--> osDelay(3500ms) - 等待 UI/ESP32 初始化
+--> UART5 IT+IDLE 中断使能
+--> 主循环:
    |
    +--> uart5_data_ready? - 等待 IDLE 中断标志
    +--> uart4_send_barcode(data, len) - 帧转发至 ESP32
    +--> xSemaphoreTake(inventory_sem, 15000ms) - 等待 SCF 结果
    |   |
    |   +--> 超时? -> printf timeout, 重新使能中断
    |   +--> 收到? -> 继续处理
    +--> 判重:遍历 inventory_item_T[28] + inventory_item_D[6]
    |   |
    |   +--> 已有(pc 匹配)? -> inventory_add_quantity() 累加数量
    |   |   +--> LED 闪烁(T 位)/电磁锁弹开(D 位)
    |   |   +--> 写历史记录 + uart4_send_history()
    |   |
    |   +--> 新品? -> 弹出选位对话框(EEZ Flow)
    |       +--> 轮询 ui_get_integer(CID) 直到用户选择
    |       +--> inventory_store_to_slot() 存入选中槽位
    |       +--> LED 闪烁/电磁锁弹开
    |       +--> uart4_send_store(loc) 上报存储位置
    |       +--> 写历史记录 + uart4_send_history()
    +--> 重新使能 UART5 中断, 清空缓冲区, 循环
  
```

2.3.2.5.3 关键输入/输出变量

变量名	类型	方向	说明
uart5_rx_buf[256]	uint8_t[]	输入	UART5 条形码接收缓冲区
uart5_data_ready	volatile uint8_t	输入	IDLE 中断标志(1=条码数据就绪)
inventory_sem	osSemaphoreId_t	同步	SCF 解码结果信号量 (UART4_RxTask 释放)
inventory_item	InventoryItem	输入	SCF 解码返回的物料信息 (pa/pc/name/type/qty)
inventory_item_T[28]	InventoryItem[]	全局	28 个 T 位物料槽(Tray, 贴片元件)
inventory_item_D[6]	InventoryItem[]	全局	6 个 D 位物料槽(Drawer, 大件元件)
history_list[32]	HistoryRecord[]	输出	操作历史记录(追加写入)

2.3.2.6 ESP32 通信模块 (UART4_RxTask)

UART4_RxTask 是 STM32N6 与 ESP32-C3 之间的通信中枢。UART4_RXNE 中断通过 ISR 回调将接收字节写入 1024 字节环形缓冲区(ring buffer), 任务从环形缓冲区读取字节送入帧解析状态机(FrameParser)。解析完成的帧通过 dispatch_table 分发至 10 个 Handler 函数处理。模块同时维护基于 NTP 的软件 RTC 时钟, 通过 ESP32 定期下发的 TIME 帧同步年月日时分秒, 再由 FreeRTOS tick 计数推算当前时间。

2.3.2.6.1 关键函数

函数名	功能描述	关键参数
UART4_RxTask()	主循环:从环形缓冲读字节->frame_parser_feed()->dispatch_frame()->分发 Handler	void *argument
UART4_RxCallback()	ISR 回调:将接收字节写入 uart4_ring 环形缓冲区	uint8_t byte
frame_parser_feed()	帧 解 析 状 态 机:WAIT_SYNC->WAIT_TYPE->WAIT_LEN->WAIT_PAYLOAD->WAIT_CHK	fp, byte
frame_parser_init()	初始化帧解析器状态为 WAIT_SYNC	FrameParser *fp
uart4_dispatch_frame()	根据帧类型查表 dispatch_table 并调用对应 Handler	type, payload, len
get_current_time_str()	基于 NTP 基准时间+FreeRTOS tick 计数的软件 RTC	buf, bufsize
uart4_send_frame()	构建帧并通过 HAL_UART_Transmit 发送至 ESP32	type, payload, len

2.3.2.6.2 流程图

```

UART4_RxTask
|
+--> osDelay(3500ms) - 等待 LVGL UI 初始化
+--> __HAL_UART_ENABLE_IT(&huart4, UART_IT_RXNE) - 使能 RXNE 中断
+--> frame_parser_init(&parser) - 初始化状态机
+--> 主循环:
    |
    +--> uart4_ring_rd != uart4_ring_wr? - 环形缓冲区有数据?
    |
    |   +--> 读 1 字节: byte = uart4_ring[rd]; rd++
    |   +--> frame_parser_feed(&parser, byte)
    |   |
    |   |   +--> STATE 0(WAIT_SYNC): 匹配 0xA5 -> STATE 1
    |   |   +--> STATE 1(WAIT_TYPE): 存 type, 初始化 chk -> STATE 2
    |   |   +--> STATE 2(WAIT_LEN): 存 len, 初始化 idx, LEN==0?->4 :->3
    |   |   +--> STATE 3(WAIT_PAYLOAD): 逐字节存 payload[idx++], 满->4
    |   |   +--> STATE 4(WAIT_CHK): 校验 chk, 匹配->返回 1(成功), 不匹配->返回-1
    |   |
    |   +--> result==1? -> uart4_dispatch_frame(type, payload, len)
    |       +--> 查 dispatch_table[10]匹配 type
    |       +--> 调用对应 Handler:
    |           handle_led / handle_drawer / handle_face_clear /
    |           handle_alert / handle_heartbeat / handle_inventory /
    |           handle_time / handle_store / handle_miniapp_outbound /
    |           handle_qty_alert
    +--> 环形缓冲空 -> osDelay(1ms) 让出 CPU

```


2.3.2.6.3 关键输入/输出变量

变量名	类型	方向	说明
uart4_ring[1024]	uint8_t[]	输入	环形接收缓冲区(ISR 写入/任务读取)
uart4_ring_wr	volatile uint16_t	输入	环形缓冲写指针(ISR 更新)
uart4_ring_rd	uint16_t	输入	环形缓冲读指针(任务更新)
parser(FrameParser)	struct	中间	帧解析状态机 (state/type/len/idx/payload[256]/chk)
dispatch_table[10]	FrameDispatchEntry[]	中间	帧类型->Handler 函数指针映射表
last_heartbeat_tick	volatile uint32_t	输入	最近心跳接收时的 FreeRTOS tick
esp32_online	volatile uint8_t	输入	ESP32 在线状态(1=在线, 0=离线)
base_year/month/day/hour/min/second	static 变量	中间	NTP 时间同步基准值
time_synced	static uint8_t	中间	时间是否已同步标志

2.3.2.7 出库执行模块 (Outbound_Task)

Outbound_Task 作为一次性执行任务(dynamic task), 遍历所有 34 个物料槽位(T 位 28 个+D 位 6 个), 检查每个 InventoryItem 的 judgment_out 标记。将所有标记为待出库的 T 位聚合成 28 位 LED 位图(Bit27 对应 T00...Bit0 对应 T27), D 位聚合成 6 位电磁锁掩码(Bit5 对应 D00...Bit0 对应 D05)。分别通过 osThreadNew 动态创建 RGBLED_Task 和 Electromagnet_Task 执行物理动作。执行完毕后清除所有 judgment_out 标记并调用 osThreadExit()自销毁。

2.3.2.7.1 关键函数

函数名	功能描述	关键参数
Outbound_Task()	遍历 34 槽位->聚合 LED 位图+ 抽屉掩码->动态创建 LED/电磁 锁子任务->清除标记->自销毁	void *argument

2.3.2.7.2 流程图

Outbound_Task (由 UI 或小程序出库触发创建)

```

|
+--> 初始化 led_bitmap=0, cab_mask=0
+--> 遍历 28 个 T 位 inventory_item_T[0..27]:
|   +--> if (judgment_out) led_bitmap |= (1u << (27-i))
+--> 遍历 6 个 D 位 inventory_item_D[0..5]:
|   +--> if (judgment_out) cab_mask |= (1u << i)
+--> if (led_bitmap != 0)
|   +--> osThreadNew(RGBLED_Task, led_bitmap) - 动态创建 LED 闪烁任务
+--> if (cab_mask != 0)
|   +--> osThreadNew(Electromagnet_Task, cab_mask) - 动态创建电磁锁任务
+--> 清除所有 judgment_out = false
+--> osThreadExit() - 任务自销毁
  
```

2.3.2.7.3 关键输入/输出变量

变量名	类型	方向	说明
inventory_item_T[28].judgment_out	bool	输入	T 位待出库标记(UI/小程序设置)
inventory_item_D[6].judgment_out	bool	输入	D 位待出库标记
led_bitmap	uint32_t	输出	28 位 LED 位图 (Bit27=T00...Bit0=T27)
cab_mask	uint8_t	输出	6 位 电 磁 锁 掩 码 (Bit5=D00...Bit0=D05)

2.3.2.8 LED 指示模块 (RGBLED_Task)

RGBLED_Task 作为一次性动态任务，接收 28 位位图参数(通过 osThreadNew argument 传递)，通过 SK9822 SPI 协议控制 28 颗 RGB LED。根据位图中置位的位将对应 LED 设为红色(R=255,G=0,B=0)闪烁，持续 6 秒后全部关闭并自销毁。SK9822 采用单线串联控制(SPI5)，支持 32 级全局亮度调节。

2.3.2.8.1 关键函数

函数名	功能描述	关键参数
RGBLED_Task()	解析位图->SK9822_SetByBitmap 红色->延时 6s->全灭->自销毁	void *argument
RGBLED_Flash()	便捷接口:动态创建 RGBLED_Task	uint32_t bitmap
SK9822_SetByBitmap()	根据位图设置 28 颗 LED 的颜色和亮度,未选中的 LED 设为全灭	bitmap,r,g,b,brightness
SK9822_Transmit()	通过 SPI5 发送 SK9822 帧数据 (32bit/颗 x28)	void

2.3.2.8.2 流程图

RGBLED_Task (动态创建，传入 bitmap 参数)

```

|
+--> bitmap = (uint32_t)(uintptr_t)argument - 解析 28 位位图
+--> SK9822_Init() - SPI5 初始化, SK9822 复位序列
+--> SK9822_SetByBitmap(bitmap, 255, 0, 0, 5)
|   +--> 遍历 28 颗 LED(i=0..27):
|       if (bitmap & (1u << (27-i))):
|           SK9822_SetGroup(i, 255, 0, 0, 5) - 红色,亮度 5/31
|       else:
|           SK9822_SetGroup(i, 0, 0, 0, 0) - 全灭
|   +--> SK9822_Transmit() - 一次 SPI 发送 28x32bit 帧
+--> vTaskDelay(6000ms) - 持续 6 秒
+--> SK9822_SetAll(0,0,0,0) + SK9822_Transmit() - 全灭
+--> osThreadExit() - 自销毁
  
```

2.3.2.8.3 关键输入/输出变量

变量名	类型	方向	说明
bitmap(uint32_t)	uint32_t	输入	28 位 LED 位图(通过 argument 传递)
hspi5	SPI_HandleTypeDef	全局	SK9822 SPI5 通信句柄(约 2.5MHz)

2.3.2.9 电磁锁控制模块 (Electromagnet)

Electromagnet 模块控制 6 路电磁锁/电控锁的 GPIO 脉冲输出。对应的 GPIO 引脚为 PG14、PB13、PQ7、PQ6、PQ5、PQ4，分别对应 6 位掩码的 Bit5~Bit0。任务接收 6 位掩码参数，依次拉高对应 GPIO，延时 500ms 后全部拉低，实现脉冲式开锁。另有 ElectromagnetLock_Task 用于锁闭控制(PQ3 脉冲)。

2.3.2.9.1 关键函数

函数名	功能描述	关键参数
Electromagnet_Task()	解析掩码->逐路 GPIO 置高->延时 500ms->全部拉低->自销毁	void *argument
Electromagnet_Open()	便捷接口：动态创建 Electromagnet_Task	uint8_t cabinet_mask
ElectromagnetLock_Task()	锁闭控制:PQ3 拉高 500ms->拉低->自销毁	void *argument

2.3.2.9.2 流程图

Electromagnet_Task (动态创建，传入 6 位掩码)

```

|
+--> cabinet_mask = (uint8_t)(uintptr_t)argument
+--> 逐路 GPIO 置高 (500ms 脉冲):
|   +--> if (mask & 0x20) HAL_GPIO_WritePin(PG14, SET)
|   +--> if (mask & 0x10) HAL_GPIO_WritePin(PB13, SET)
|   +--> if (mask & 0x08) HAL_GPIO_WritePin(PQ7, SET)
|   +--> if (mask & 0x04) HAL_GPIO_WritePin(PQ6, SET)
|   +--> if (mask & 0x02) HAL_GPIO_WritePin(PQ5, SET)
|   +--> if (mask & 0x01) HAL_GPIO_WritePin(PQ4, SET)
+--> vTaskDelay(500ms) - 脉冲宽度
+--> 全部拉低:
|   HAL_GPIO_WritePin(PG14, RESET); ...
|   HAL_GPIO_WritePin(PQ7|PQ6|PQ5|PQ4, RESET)
+--> osThreadExit()

```

2.3.2.9.3 关键输入/输出变量

变量名	类型	方向	说明
cabinet_mask	uint8_t	输入	6 位电磁锁掩码 (Bit5=PG14, Bit4=PB13, Bit3=PQ7, Bit2=PQ6, Bit1=PQ5, Bit0=PQ4)
GPIOG_PIN_14	GPIO	输出	电磁锁 0 (掩码 Bit5)
GPIOB_PIN_13	GPIO	输出	电磁锁 1 (掩码 Bit4)
GPIOQ_PIN_7	GPIO	输出	电磁锁 2 (掩码 Bit3)
GPIOQ_PIN_6	GPIO	输出	电磁锁 3 (掩码 Bit2)
GPIOQ_PIN_5	GPIO	输出	电磁锁 4 (掩码 Bit1)
GPIOQ_PIN_4	GPIO	输出	电磁锁 5 (掩码 Bit0)

2.3.2.10 温湿度采集模块 (Aht10_Task)

Aht10_Task 通过软件模拟 I2C(GPIOH_PIN7=SDA, GPIOH_PIN8=SCL)与 AHT10 传感器通信, 以 2 秒为周期读取环境温湿度数据。每次读取后通过 UART4 帧协议上发至 ESP32(MQTT 云平台), 同时更新 EEZ Flow 全局变量(TEM/HUM)刷新 UI 仪表盘显示。若 AHT10 初始化失败, 设置 Cabinet.Aht10_state=false 并挂起自身任务。

2.3.2.10.1 关键函数

函数名	功能描述	关键参数
Aht10_Task()	AHT10 初始化->2s 周期读取->UART4 上报+UI 更新	void *argument
aht10_init()	软件 I2C 初始化+AHT10 校准命令(0xE1, 0x08, 0x00)	void
aht10_read()	触发测量(0xAC, 0x33, 0x00)->等待 75ms->读取 6 字节->解析温湿度	float *h, float *t
uart4_send_temperature()	构建 TEMP 帧 (FRAME_TYPE_TEMP) 发送至 ESP32	float val
uart4_send_humidity()	构建 HUMID 帧(FRAME_TYPE_HUMID)发送至 ESP32	float val

2.3.2.10.2 流程图

```

Aht10_Task
|
+--> osDelay(3500ms) - 等待 LVGL UI 初始化
+--> aht10_init()
|   +--> 失败? -> goto error: Cabinet.Aht10_state=false, vTaskSuspend(NULL)
+--> Cabinet.Aht10_state = true
+--> 主循环 (2s 周期):
    +--> aht10_read(&humidity, &temperature)
    +--> uart4_send_temperature(temperature) - UART4 帧上报
    +--> uart4_send_humidity(humidity)
    +--> ui_set_integer(TEM, (int)temperature) - UI 更新
    +--> ui_set_integer(HUM, (int)humidity)
    +--> osDelay(2000ms)
  
```

2.3.2.10.3 关键输入/输出变量

变量名	类型	方向	说明
Cabinet.temperature	float	输出	当前温度(摄氏度)
Cabinet.humidity	float	输出	当前湿度(百分比)
Cabinet.Aht10_state	bool	输出	AHT10 传感器状态(true=正常)
FLOW_GLOBAL_VARIABLE_TEM(7)	int	输出	EEZ Flow 温度显示变量
FLOW_GLOBAL_VARIABLE_HUM(8)	int	输出	EEZ Flow 湿度显示变量

2.3.2.11 接近传感模块 (VL53L1X_Task)

VL53L1X_Task 通过 I2C2 接口控制 VL53L1X ToF 激光测距传感器, 实现接近检测与屏幕背光自动控制。传感器配置为长距离模式(最远 4m), 100ms 测量周期, 连续测距。模块实现双阈值+滞回状态机:当人员远离(distance > 400mm)持续 5 秒后关闭屏幕背光进入节能模式, 当人员靠近(distance <= 400mm)持续 1 秒后开启背光恢复正常显示。

2.3.2.11.1 关键函数

函数名	功能描述	关键参数
VL53L1X_Task()	传感器初始化->连续测距->双阈值状态机 ->背光控制	void *argument
vl53l1x_boot_state()	等待传感器启动完成	uint8_t *state
vl53l1x_get_sensor_id()	读取传感器 ID 验证(应为 0xEACC)	uint16_t *id
vl53l1x_init()	VL53L1X 初始化配置	void
vl53l1x_set_distance_mode()	设置测距模式(2=长距离)	uint8_t mode
vl53l1x_set_timing_budget_ms()	设置测量周期(100ms)	uint32_t ms
vl53l1x_start_ranging()	启动连续测距模式	void
vl53l1x_read_distance()	读取当前距离值(mm)	uint16_t *dist
RGBLCD_BL()	控制 LCD 背光开关	uint8_t on_off

2.3.2.11.2 流程图

```

VL53L1X_Task
|
+-- 等待传感器启动: vl53l1x_boot_state() 循环
+-- vl53l1x_get_sensor_id() - 验证 ID==0xEACC
+-- vl53l1x_init() + set_distance_mode(2) + set_timing_budget_ms(100)
+-- vl53l1x_start_ranging() - 连续测距
+-- proximity_state=0(默认=近), state_enter_tick=tick
+-- 主循环:
|
|   +-- vl53l1x_is_data_ready() - 等待测量数据就绪
|   +-- vl53l1x_read_distance() - 读取距离
|   +-- vl53l1x_clear_interrupt() - 清除中断
|   +-- distance > threshold(400mm)?
|   |   +-- YES (远离):
|   |   |   proximity_state==0? -> 进入 far 状态, 记录 tick
|   |   |   elapsed>=5000ms? -> System_state=false, RGBLCD_BL(0) 关闭背光
|   |   +-- NO (靠近):
|   |   |   proximity_state==1? -> 进入 near 状态, 记录 tick
|   |   |   elapsed>=1000ms? -> System_state=true, RGBLCD_BL(1) 开启背光
|   +-- vTaskDelay(20ms)

```

2.3.2.11.3 关键输入/输出变量

变量名	类型	方向	说明
Cabinet.distance_mm	uint16_t	输出	当前测量距离(mm)
Cabinet.distance_mm_th	uint16_t	输入	距离阈值(400mm,默认值)
Cabinet.VL53L1X_state	bool	输出	传感器状态(true=正常工作)
Cabinet.System_state	bool	输出	系统激活状态(false=休眠节能)
proximity_state	uint8_t	中间	接近状态(0=近,1=远,用于滞回)
state_enter_tick	uint32_t	中间	进入当前接近状态的时间戳

2.3.2.12 打印模块 (Printer_Task)

Printer_Task 控制热敏打印机通过 UART7(TTL 串口)输出 BOM 物料领用清单标签。模块封装了打印机指令集:汉字放大、加粗、居中对齐、打印虚线、走纸、切纸等。打印内容使用 GBK 编码的汉字字节数组，包含表头(近期物料领用清单)、基础信息区(单号/时间/领用人)、物料列表区(名称/规格/数量/状态)以及结束动作(走纸 4 行+切纸)。

2.3.2.12.1 关键函数

函数名	功能描述	关键参数
Printer_Task()	调用 Print_Recent_BOM()->进入空闲循环	void *argument
Print_Recent_BOM()	完整的BOM标签打印流程:初始化->表头->信息->物料列表->切纸	void
Printer_Init()	打印机初始化(复位+清空格式缓存)	void
Set_ChineseFontSize()	设置汉字字体大小(0=正常,1=放大)	uint8_t size
Set_FontStyle()	设置字体样式(加粗/下划线/反白)	uint8_t bold, ul, inv
Set_Alignment()	设置对齐方式(0=左,1=中,2=右)	uint8_t align
Print_Line()	打印虚线分割线	void
Print_Text()	打印 ASCII 文本	const char *text
Printer_Feed()	走纸 n 行	uint8_t lines
Printer_Cut()	切纸(1=全切, 0=半切)	uint8_t mode

2.3.2.12.2 流程图

```

Printer_Task
|
+--> Print_Recent_BOM()
|
+--> Printer_Init() - 初始化并清空格式
+--> [表头区]
|   +--> 居中对齐 + 汉字放大 + 加粗
|   +--> 打印 GBK 标题: 近期物料领用清单
+--> [基础信息区]
|   +--> 恢复正常大小 + 取消加粗 + 左对齐
|   +--> 单号: BOM-20260622-001
|   +--> 时间: 2026-06-22 20:25
|   +--> 领用人: 硬件开发组
+--> Print_Line() - 虚线分割
+--> [物料列表区]
|   +--> 表头: 物料名称/规格 + 数量 + 状态
|   +--> 分隔线: ---
|   +--> 逐项打印 (GBK 汉字):
|       贴片电阻 0603 10K    x500    已出库
|       贴片电容 0402 1uF    x200    已出库
|       STM32 芯片 G431RBT    x5      已出库
|       排针 2.54mm 1x40P    x10     已出库
|       红色 LED 0805         x50     已出库
+--> Print_Line()
+--> [结束动作]
|   +--> Printer_Feed(4) - 走纸 4 行
|   +--> Printer_Cut(1) - 全切纸
+--> 主循环: vTaskDelay(10ms) 空闲
    
```

2.3.2.12.3 关键输入/输出变量

变量名	类型	方向	说明
huart7(UART7)	UART_HandleTypeDef	全局	热敏打印机 TTL 串口句柄(GBK 编码)
title/item[]	uint8_t[]	输入	GBK 编码的打印内容字节数组

2.3.2.13 环境预警模块 (handle_alert + handle_qty_alert)

环境预警功能由两个 ESP32 下行帧 Handler 实现。handle_alert 处理温湿度阈值告警:微信小程序下发预警阈值(TEMP_TH/HUMID_TH)后,ESP32通过MQTT接收并以ALERT帧(0x04)转发至 STM32, STM32 更新 Cabinet.temperature_th/humidity_th 并刷新 UI 预警计数。handle_qty_alert 处理库存预警:ESP32 检测到物料库存低于安全线时下发 QTY_ALERT 帧(0x0A), STM32 将预警信息填充至 alarm_list 并通过 ui_push_alarm 同步到 EEZ Flow UI 显示。

2.3.2.13.1 关键函数

函数名	功能描述	关键参数
handle_alert()	解析 ALERT 帧(sub_type+val)->更新温湿度阈值->UI 刷新	payload, len
handle_qty_alert()	解析 QTY_ALERT 帧(pc+cab+qty)->填充 alarm_list->UI 推送	payload, len
ui_push_alarm()	将 C 语言 AlarmRecord 数组转换为 EEZ Flow ArrayOfLSWValue 并设置全局变量	list, count

2.3.2.13.2 流程图

```

ESP32 MQTT 订阅 cmd001 -> JSON 解析 -> UART4 帧转发 -> STM32
|
+--> 温湿度阈值告警 (ALERT 0x04):
|   +--> payload: sub_type[1B] + raw_val[2B]BE
|   +--> sub_type==0x00: Cabinet.temperature_th = val*0.1
|   +--> sub_type==0x01: Cabinet.humidity_th = val*0.1
|   +--> ui_set_integer(DASH_TS, DASH_TS+1) - UI 预警计数+1
|
+--> 库存预警 (QTY_ALERT 0x0A):
    +--> payload: pc_str\0 + cab[1B] + qty[2B]BE
    +--> 填充 alarm_list[0] = {pc, cabinet_id, quantity}
    +--> ui_push_alarm(alarm_list, 1) - EEZ Flow LSW 变量更新
        +--> osMutexAcquire(flow_var_mutex) - 互斥保护
        +--> ArrayOfLSWValue: name=pc, CNUM=cab_id, STOCK=qty
        +--> setGlobalVariable(FLOW_GLOBAL_VARIABLE_LSW, lsws)
        +--> osMutexRelease(flow_var_mutex)

```

2.3.2.13.3 关键输入/输出变量

变量名	类型	方向	说明
Cabinet.temperature_th	float	输出	温度预警阈值(小程序下发)
Cabinet.humidity_th	float	输出	湿度预警阈值(小程序下发)
FLOW_GLOBAL_VARIABLE_DASH_TS(5)	int	输出	UI 预警总次数计数
alarm_list[1]	AlarmRecord[]	输出	库存预警记录(pc/cabinet_id/quantity)
FLOW_GLOBAL_VARIABLE_LSW(28)	Array	输出	EEZ Flow 预警列表显示变量

2.3.2.14 UI 桥接层 (ui_bridge.cpp)

ui_bridge.cpp 是 C/C++ 应用代码与 EEZ Flow UI 框架之间的数据桥接层。提供类型安全的 getter/setter 接口(ui_set_integer, ui_get_integer, ui_set_string, ui_get_string) 用于读写 29 个 EEZ Flow 全局变量。核心功能包括:InventoryItem 数组与 CabinetValue 数组的双向转换(ui_push_inventory/ui_pull_inventory), HistoryRecord 数组与 LogEntryValue 数组的转换(ui_push_history/ui_pull_history), 以及 AlarmRecord 与 LSWValue 的转换(ui_push_alarm)。对 flow_var_mutex 保护的全局变量写操作使用互斥锁确保线程安全。

2.3.2.14.1 关键函数

函数名	功能描述	关键参数
ui_set_integer()	设置 EEZ Flow 全局变量(整数类型)	var_id, value
ui_get_integer()	读取 EEZ Flow 全局变量(整数类型)	var_id
ui_set_string()	设置 EEZ Flow 全局变量(字符串类型)	var_id, value
ui_get_string()	读取 EEZ Flow 全局变量(字符串类型)	var_id
ui_push_inventory()	将 InventoryItem 数组(28T+6D)转为 CabinetValue 数组推送到 UI	t_items, t_count, d_items, d_count
ui_pull_inventory()	从 UI 拉取 CabinetValue 数组转回 InventoryItem 数组	t_items, t_count, d_items, d_count
ui_push_history()	将 HistoryRecord 数组转为 LogEntryValue 数组推送到 UI	list, count
ui_pull_history()	从 UI 拉取 LogEntryValue 数组转回 HistoryRecord 数组	list, max_count
ui_push_alarm()	将 AlarmRecord 数组转为 LSWValue 数组推送到 UI(互斥保护)	list, count

2.3.2.14.2 流程图

```

ui_bridge 数据流
|
+--> C/C++代码 (任务层)
|   |
|   +--> InventoryItem[34] (T 位 28 + D 位 6)
|   +--> HistoryRecord[32]
|   +--> AlarmRecord[1]
|   +--> 基础类型变量 (TEM, HUM, DASH_CTG, DASH_LS, ...)
|
+--> ui_bridge.cpp (桥接层)
|   |
|   +--> copy_item_to_cab(): InventoryItem -> CabinetValue (name/qty/pa/spec/pc)
|   +--> copy_cab_to_item(): CabinetValue -> InventoryItem
|   +--> ArrayOfCabinetValue: EEZ Flow 数组类型封装
|   +--> ArrayOfLogEntryValue: 历史记录数组
|   +--> ArrayOfLSWValue: 库存预警数组
|   +--> osMutexAcquire/Release(flow_var_mutex) - 数据一致性保护
|
+--> EEZ Flow UI (显示层)
|   |
|   +--> 29 个全局变量 (FLOW_GLOBAL_VARIABLE_*)
|   +--> 4 个 UI 页面: Lock / Camera / Dashboard / Selection
|   +--> 数据绑定: 变量变化自动刷新控件

```

2.3.2.14.3 关键输入/输出变量

变量名	类型	方向	说明
FLOW_GLOBAL_VARIABLE_CABINETS(10)	ArrayOfCabinetValue	双向	所有物料槽位数组(34 项)
FLOW_GLOBAL_VARIABLE_CABINET_COUNT(14)	int	输出	已占用槽位数
FLOW_GLOBAL_VARIABLE_HISTORY_RECORDS(16)	ArrayOfLogEntryValue	双向	操作历史记录数组
FLOW_GLOBAL_VARIABLE_LSW(28)	ArrayOfLSWValue	输出	库存预警列表
FLOW_GLOBAL_VARIABLE_TEMP_NAME(19)	string	输出	新物料名称(选位对话框)
FLOW_GLOBAL_VARIABLE_TEMP_PC(21)	string	输出	新物料 PC 编号
FLOW_GLOBAL_VARIABLE_TEMP_PA(22)	string	输出	新物料 PA 编号
FLOW_GLOBAL_VARIABLE_TEMP_SPEC(23)	string	输出	新物料规格型号
FLOW_GLOBAL_VARIABLE_CID(25)	int	输入	用户选择的槽位 ID(-1=未选择)
FLOW_GLOBAL_VARIABLE_SHOW_SELECT_DIALOG(24)	int	输出	选位对话框显示控制

2.3.2.15 ESP32-C3 桥接固件 (sketch_jun26a.ino)

ESP32-C3 固件采用 Arduino 框架开发，作为 STM32N647 与云端的 WiFi 通信桥梁。固件在 setup()中初始化 WiFi STA 连接、Bemfa MQTT 客户端(bemfa.com:9501)、UART 串口(115200bps)。loop()主循环中执行 5 大功能块:WiFi 重连维护、MQTT 重连+订阅恢复、UART 帧接收+转发(STM32 上行帧转 MQTT 发布，MQTT 下行消息转 UART 帧发送)、NTP 时间同步(每小时)+心跳包(每 15 秒)、以及腾讯云 SCF HTTPS 请求(扫码解码)。UART 接收采用帧解析状态机(与 STM32 侧对称)，MQTT 下行通过 cmd001 主题订阅接收 JSON 指令。

2.3.2.15.1 关键函数

函数名	功能描述	关键参数
setup()	WiFi 连接+MQTT 初始化+NTP 校时+UART 初始化	void
loop()	主循环:WiFi 维护/MQTT 维护/UART 帧处理/心跳+时间/SCF 请求	void
mqtt_loop()	MQTT 客户端 keep-alive+消息接收处理	void
uart_loop()	UART 帧解析+STM32 上行帧转 MQTT 发布	void
send_http_scf()	HTTPS POST 条形码到腾讯云 SCF->解析 JSON->回复 STM32	barcode_data
parse_cmd001()	解析 MQTT cmd001 订阅消息(JSON)->转 UART 帧下发 STM32	json_payload

2.3.2.15.2 流程图

```

ESP32-C3 setup()
|
+--> Serial.begin(115200) - UART0 调试输出
+--> Serial2.begin(115200, ...) - UART2 连接 STM32 UART4
+--> WiFi.begin(ssid, password) - STA 模式连接路由器
+--> mqtt_client.setServer(bemfa.com, 9501)
+--> mqtt_client.setCallback(mqtt_callback) - 注册 cmd001 订阅回调
+--> mqtt_connect() + subscribe(cmd001)
+--> configTime() - NTP 时间同步
|
ESP32-C3 loop()
|
+--> WiFi.status() != WL_CONNECTED? -> WiFi.reconnect()
+--> !mqtt_client.connected()? -> mqtt_connect() + resubscribe
+--> mqtt_client.loop() - MQTT keep-alive + 消息处理
+--> UART2.available()? -> 帧解析状态机:
|   +--> 接收 STM32 上行帧(0x11~0x1A)
|   +--> 根据帧类型转发:
|       TEMP(0x11) -> mqtt.publish(temp, val)
|       HUMID(0x12) -> mqtt.publish(humid, val)
|       BARCODE(0x18) -> send_http_scf(data) -> 等 HTTPS 响应
|                               -> 构建 INVENTORY(0x06) 帧回传 STM32
|       HISTORY(0x1A) -> mqtt.publish(history, json)
|       ... (其他上行帧同理)
+--> 心跳计时: 每 15s 发送 HEARTBEAT(0x05) 帧至 STM32
|   payload[0] = (WiFi_OK << 0) | (MQTT_OK << 1)
+--> NTP 计时: 每小时 configTime() 同步一次
|   +--> 构建 TIME(0x08) 帧下发 STM32
+--> mqtt_callback (cmd001 消息到达):
|   +--> 解析 JSON 指令 {cmd, cabinet, qty, ...}
|   +--> 转换对应 UART 帧:
|       LED(0x01) / DRAWER(0x02) / MINIOUT(0x09) / ...
+--> uart2_send_frame() 下发 STM32

```

2.3.2.15.3 关键输入/输出变量

变量名	类型	方向	说明
WiFi.status()	uint8_t	状态	WiFi 连接状态(WL_CONNECTED=已连接)
mqtt_client.connected()	bool	状态	MQTT 连接状态
Serial2(UART2)	HardwareSerial	全局	与 STM32 UART4 连接的串口(115200bps)
mqtt_client(WiFiClient)	PubSubClient	全局	Bemfa MQTT 客户端实例
barcode_data	String	中间	条码原始数据(HTTPS POST 至 SCF)
scf_response	String	中间	腾讯云 SCF JSON 响应(pa/pc/name/type/qty)
cmd001_payload	String	中间	MQTT 下行订阅消息 JSON

2.3.2.16 SCF 云端解码流程

当 STM32 通过扫码枪获取条形码原始数据后，由 ESP32-C3 通过 HTTPS POST 请求发送至腾讯云 SCF(Serverless Cloud Function)进行 AI 解码。SCF 函数利用 OCR/条码识别 AI 模型对原始数据解码，提取物料信息(pa=位置编码, pc=产品编码, name=物料名称, type=规格型号, quantity=数量)。解码结果以 JSON 格式返回 ESP32-C3, ESP32 解析后构建 INVENTORY 帧(0x06)通过 UART4 下发 STM32，同时通过 MQTT inv001 主题发布解码结果到云平台供小程序查看。

2.3.2.16.1 关键函数

函数名	功能描述	关键参数
send_http_scf()	构建 HTTPS POST 请求(JSON body)->发送至 SCF URL->等待响应->解析 JSON	barcode_raw
(SCF 函数)	接收条码原始数据->AI OCR/解码->提取 {pa,pc,name,type,qty}->返回 JSON	event, context

2.3.2.16.2 流程图

扫码入库 -> SCF 云端解码流程

```

|
+--> 扫码枪 USB 串口 -> UART5 -> STM32 Barcode_Task
+--> STM32 -> UART4 帧(0x18 BARCODE) -> ESP32-C3
+--> ESP32-C3 uart_loop() 收到 BARCODE 帧:
|
|   |
|   +--> 提取 barcode_raw_data (UART 帧 Payload)
|   +--> send_http_scf(barcode_raw_data)
|       |
|       +--> 构建 HTTPS POST 请求:
|           |
|           |   URL: https://service-xxx.apigw.tencentcs.com/release/barcode
|           |   Header: Content-Type: application/json
|           |   Body: {"barcode": "<raw_data>"}
|           +--> HTTPClient.POST() -> 等待响应
|       +--> 解析 JSON 响应:
|           |
|           |   {
|           |       "pa": "A01",
|           |       "pc": "RES-0603-10K",
|           |       "name": "贴片电阻",
|           |       "type": "0603 10K 5%",
|           |       "quantity": 500
|           |   }
|       +--> 构建 INVENTORY 帧(0x06):
|           |
|           |   payload = pa\0 pc\0 name\0 type\0 qty[2B]BE src[1B]
|           |   src = 0x00 (来源=STM32 扫码)
|       +--> uart2_send_frame(0x06, payload, len) -> STM32
|       +--> mqtt_client.publish("inv001", json_str) -> Bmf Cloud
|
+--> STM32 UART4_RxTask 收到 INVENTORY 帧:
    +--> handle_inventory() 解析 payload 填充 inventory_item
    +--> on_inventory_stm32() -> xSemaphoreGive(inventory_sem)
    +--> Barcode_Task 被信号量唤醒 -> 判重入库流程

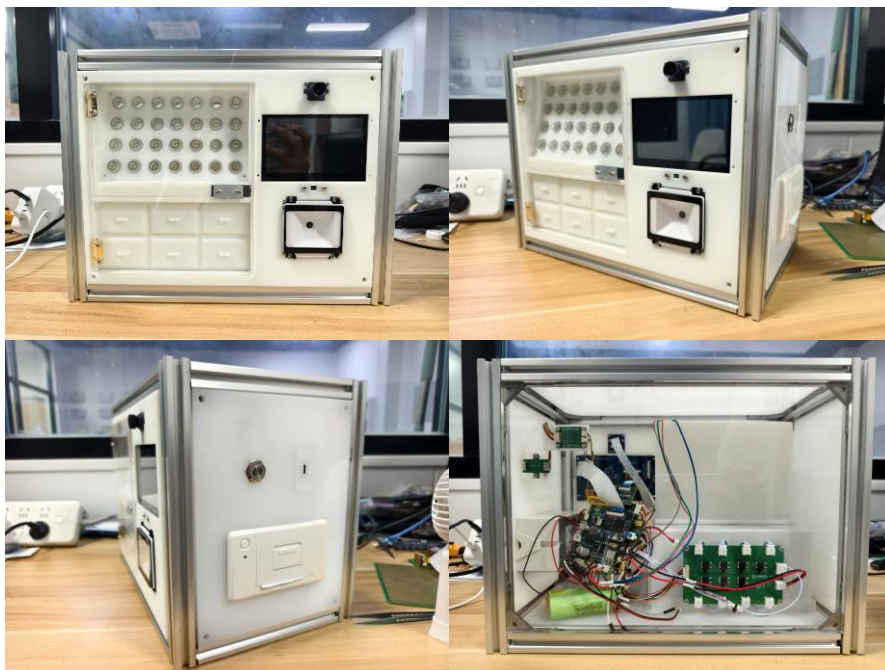
```

2.3.2.16.3 关键输入/输出变量

变量名	类型	方向	说明
barcode_raw	String	输入	扫码枪原始条形码数据(ASCII/二进制)
scf_response_json	String	输出	SCF 解码结果 JSON(pa/pc/name/type/quantity)
inventory_item	InventoryItem	输出	解析后的物料信息结构体
FRAME_TYPE_INVENTORY(0x06)	帧	输出	SCF 结果回传 STM32 的帧类型
FRAME_TYPE_BARCODE(0x18)	帧	输入	STM32 请求 SCF 解码的帧类型
inv001(MQTT Topic)	Topic	输出	SCF 解码结果 MQTT 发布主题

第三部分 完成情况及性能参数

3.1 整体介绍

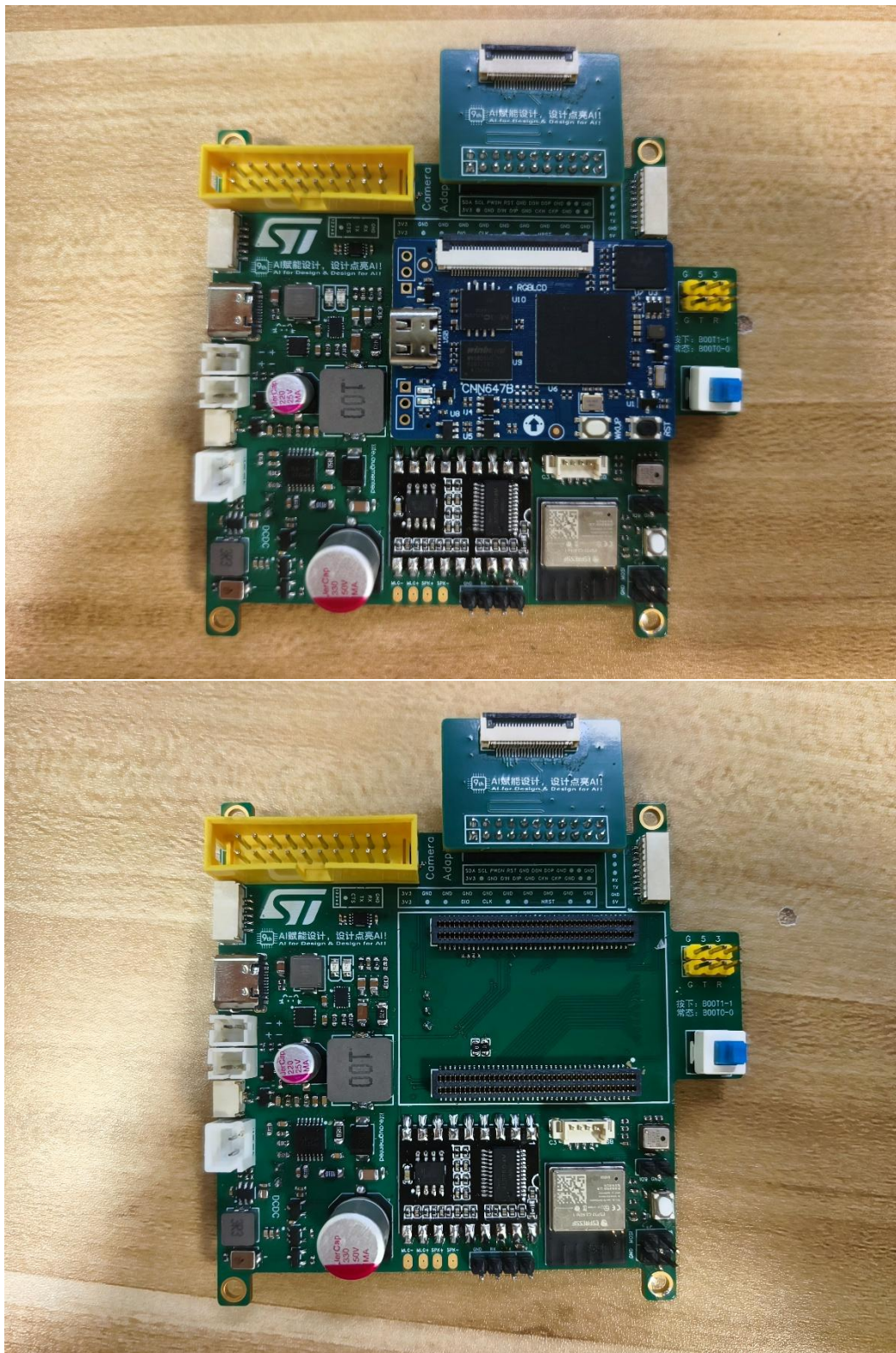


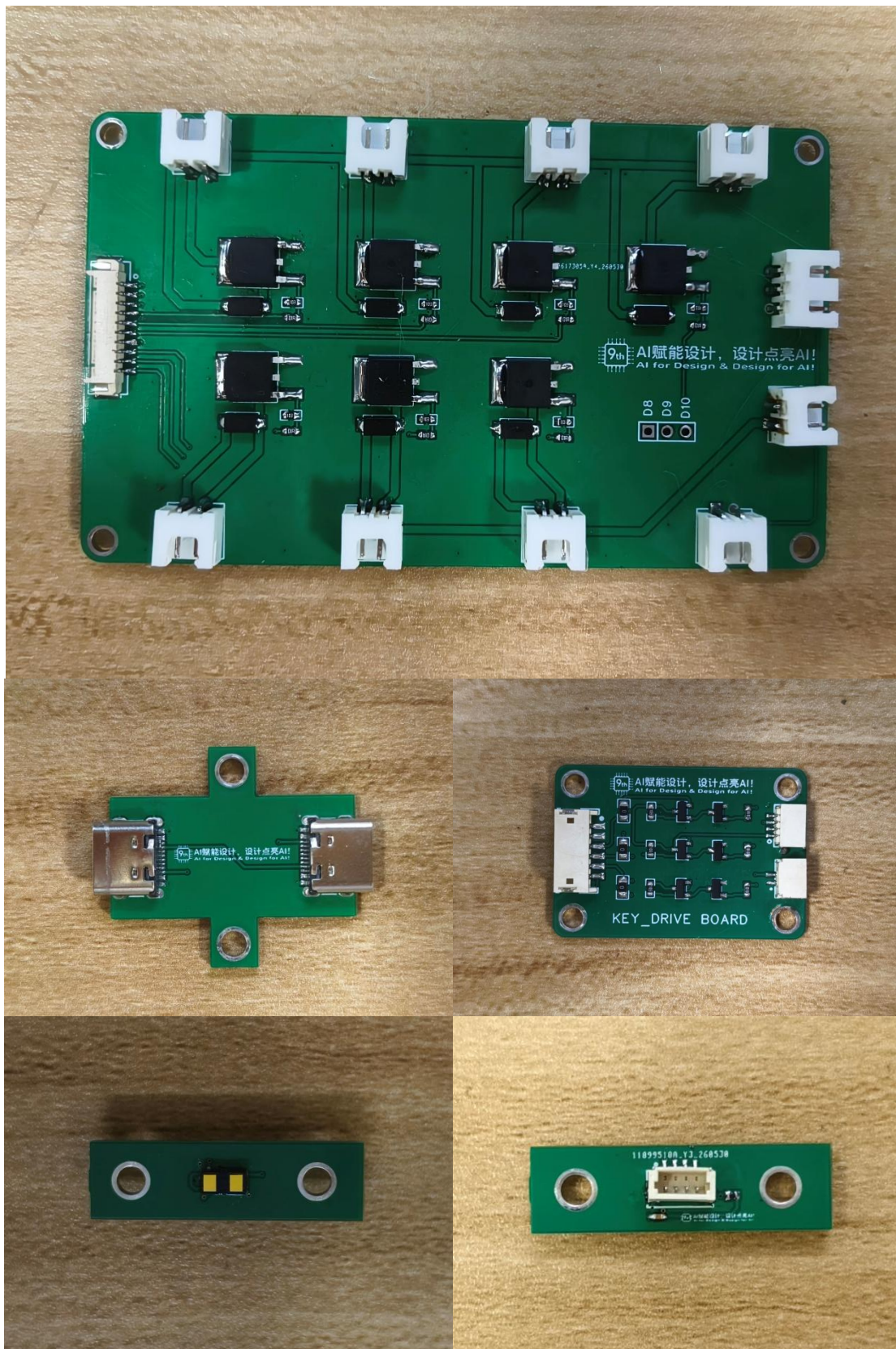
3.2 工程成果（分硬件实物、软件界面等设计结果）

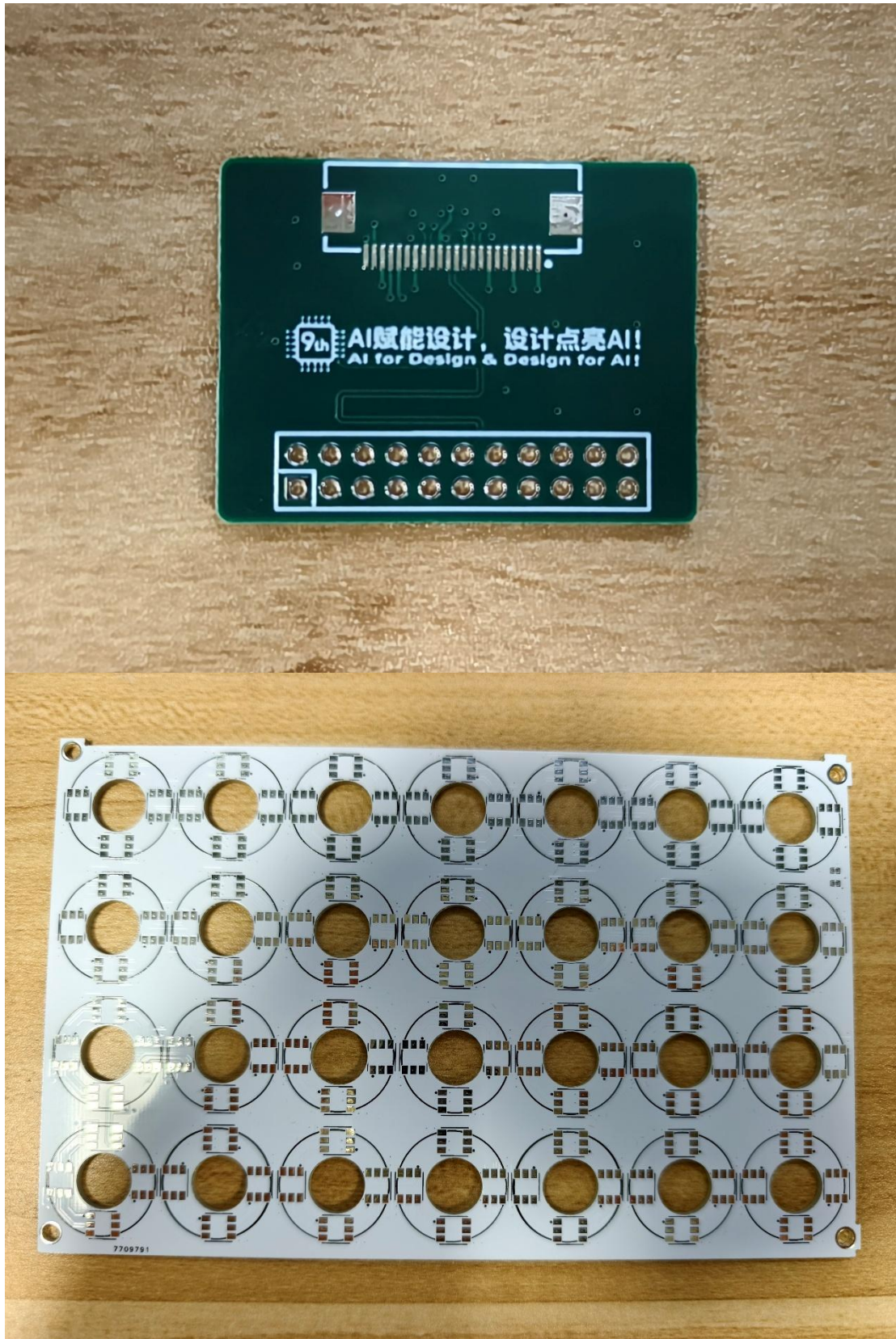
3.2.1 机械成果；

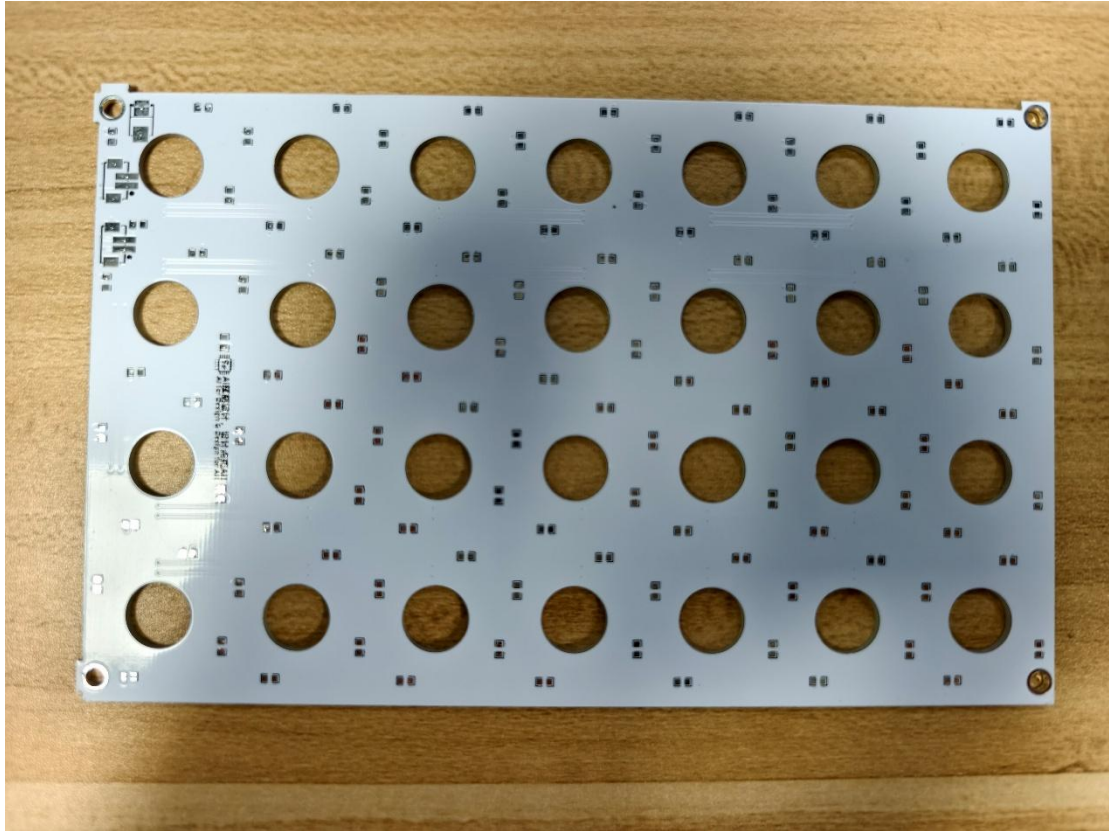


3.2.2 电路成果： 图中除蓝色的正点原子 N6 核心板外均为团队自主设计



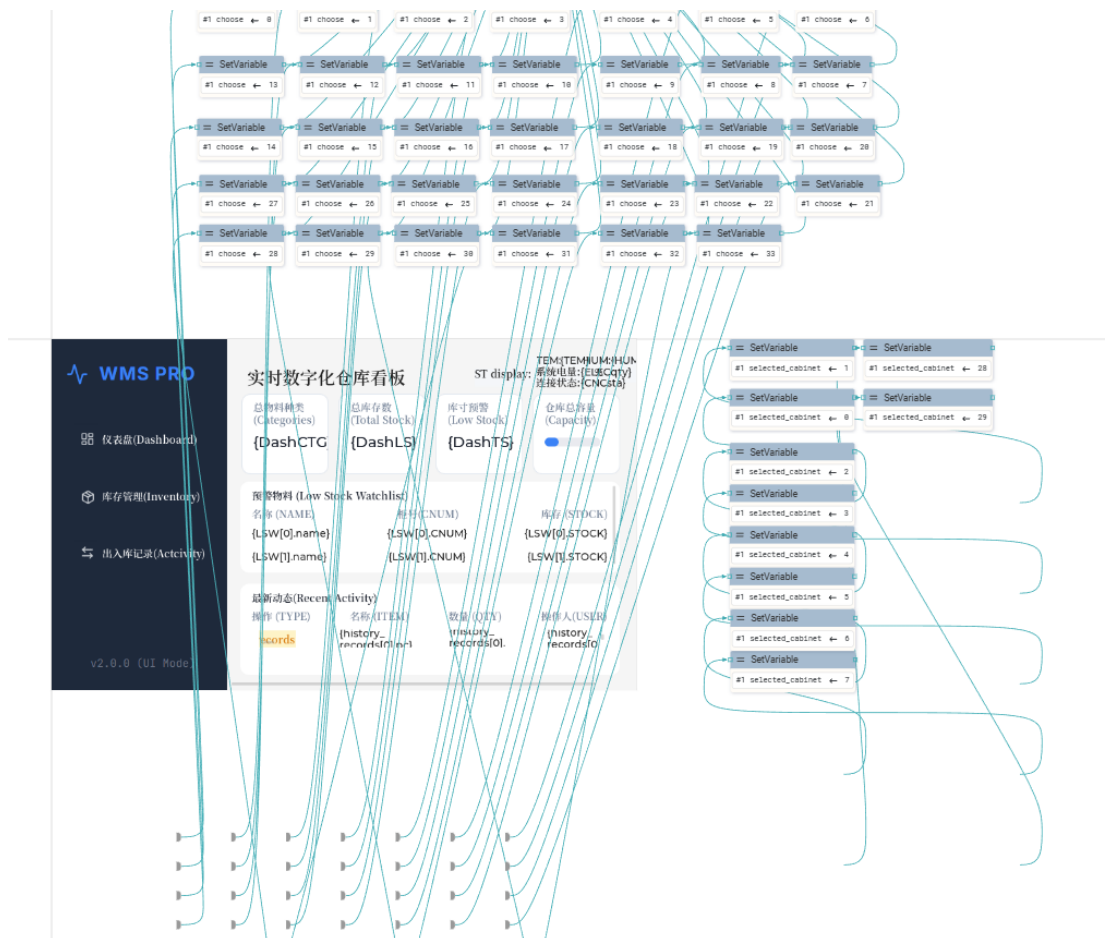






3.2.3 软件成果：





3.3 特性成果



第四部分 总结

4.1 可扩展之处

- ◆ 人脸库现在存本地，后续可上云做多设备同步
- ◆ 条形码识别依赖腾讯云 SCF，可本地部署轻量 OCR 模型脱机运行
- ◆ 34 个槽位固定，可改成动态槽位管理适应不同尺寸元器件

- ◆ MQTT 走 Bemfa 免费平台，量大可迁自建 EMQX
- ◆ 预留了 I2C/SPI 口，温湿度之外可加速测、称重传感器
- ◆ LVGL 就 4 个页面，缺数据统计页、历史曲线页
- ◆ 打印模板写死了 BOM 格式，改成模板可配会更灵活

4.2 心得体会

从零开始把两个 AI 模型塞进 STM32N6 这块 NPU 开发板，前后调试了快一个月。现在回头看那些 .md 调试记录，最大的感触不是技术本身，而是一个很朴素的道理：嵌入式 AI 的坑不在模型，在硬件资源的地狱级博弈。

一、CPU / NPU / LTDC / DCMIPP 全挤在总线上抢饭吃

这大概是贯穿全程的主线剧情。一开始双图层 RGB565+RGB888 方案，LTDC 每两个像素时钟就要在 HyperRAM 的两个地址区切换一次，HyperBus 重新寻址的开销干翻了 FIFO——花屏。改单层 RGB888 后带宽够用了，但 NPU 一跑人脸识别模型（fc），直接把激活数据写到了 HyperRAM 0x90000000 开头，正好把同一地址的 LCD 显存覆盖得干干净净。屏幕雪花里能看到人影晃动——因为 DCMIPP 还在往那儿写新帧，局部覆盖了被 NPU 破坏的数据。

后来改 linker 脚本把显存挪到 0x90200000，map 确认布局正确，烧录进去雪花依旧。说明 NPU 的实际写入范围超出了编译器报告的 0x90188000，或者 cache writeback 波及了后面的区域。这事到现在还没彻底解决。

教训：在 STM32N6 这种异构多主设备架构上，内存布局不是“分好就行”，而是“分好之后还得在硬件上验证到底有没有人越界”。编译器报告的内存使用只是给 AI 编译器看的，不是给总线看的。

二、代码生成器给的“正确”代码，拆开全是桩

EEZ-Flow 是 ST 官方推荐的 LVGL UI 设计工具，画完界面一键导出 C++ 代码。生成出来的 eez-flow.cpp 一万多行，编译零 warning，跑起来也没报错。直到给历史记录列表加了数据绑定——UI 突然卡死，串口打出一行 assertion “false” failed: file “eez-flow.cpp”, line 6387。

顺着调用链追下去，发现 EEZ-Flow 生成的代码里有一个 stopScript() 函数，实现就一行 assert(false)。它是 Flow 脚本执行完毕后的清理钩子，工程里之前只有纯数据绑定的列表（不触发 Flow），所以这行桩代码躺了一个多月都没人踩到。

历史记录列表是第一个触发了 Flow 执行的功能——Flow 跑完 → 调 stopScript() → assert(false) → LV_Task 线程死在死循环里，屏幕冻结。

问题不在于这行 assert 本身，而在于你不知道还有多少类似的桩散落在那一万多行生成代码里。showKeyboardHook、showKeypadHook，这些钩子默认全部是 assert(false)。工具告诉你"代码生成成功"，但没说"其中有 5 个函数需要你自已实现否则会炸"。

这跟 DCMIPP 管道的问题本质上是一回事。DCMIPP 在连续采集模式下，调用 HAL_DCMIPP_CSI_PIPE_Stop 返回值是 HAL_OK，但物理层的 CSI PHY 和像素管道 FIFO 并没有真正复位。下次再 Start，HAL 软件状态机正确切过去了，硬件状态机还卡在上次停止的中间态——画面冻结。最后干脆放弃停止/重启的思路，让 Sensor_Task 创建一次常驻运行，摄像头界面切换只控制 LTDC 的 Color Keying 开关，不碰 DCMIPP 硬件。

教训：自动生成的代码、HAL 的返回值，都只保证"软件层面走完了"，不保证硬件层面真的做了你说的事。在嵌入式 AI 场景里，每一个跨硬件模块的操作——DCMIPP 的管道启停、NPU 的 epoch 调度、NOR Flash 的权重加载——都得当成黑盒来审视，不能因为 SDK 返回 OK 就当它 OK。

三、模型量化路上的连环翻车

- YuNet 模型四轮排查确认 int8 量化损坏，模型不可用，白费两天。

- BlazeFace int8 量化遇到 NOR Flash 权重未更新，推理坐标偏差严重；又遇到 110/183 个 epoch 在 CPU 软件回退（全是 PRelu + Dequantize + Quantize 三元组），NPU 利用率大打折扣。

- MobileFaceNet 试过 float16 量化，N6 NPU 直接不支持，思路排除。

教训：模型在 PC 端跑得好，不代表能在 NPU 上跑对。量化前后的精度损失、算子是否被 NPU 硬件支持、权重烧录是否正确，每一步都可能翻车。这块没有捷径，就是一个一个模型试。

四、一些心里话

写调试记录的时候没想过会变成心得体会的素材。当时就是踩一个坑记一个，怕忘了。现在串起来看，最大的收获不是学会了某个具体 API 的用法，而是慢慢养成了一种分层剥离的调试本能：从 HAL → LL 寄存器 → 纯 GPIO → 换

引脚，从软件到硬件一层层剥开。嵌入式 AI 的问题从来不会停留在某一层，"两个任务都能单独跑，放一起就死"才是常态。

另外有一个很实际的感悟：工具链的成熟度直接决定了你能走多远。X-CUBE-AI 的 Neural-ART 编译器虽然没有开源框架那套灵活，但在 ST 生态内至少能保证 NPU 调度图是正确的、权重地址不会自己跑偏。但一旦跨出 ST 官方验证过的场景——比如双模型权重分开放两块 NOR Flash 区间、DCMIPP 双管道同时跑显示和推理——工具的边界就暴露了。验证过的 Demo 和你的实际工程之间的差距，才是真正的开发成本。这个差距目前只能靠人肉填。

第五部分 参考文献

均为团队技术参考的贴子

<https://wiki.alientek.com/docs/Boards/STM32/DNN647/TOC>

<https://community.st.com/t5/stm32-mcus/how-to-debug-stm32n6-using-stm32cubeide/ta-p/800547>

https://mp.weixin.qq.com/s/AOJDc9-awpjl7-itBfoE_w

<https://community.st.com/t5/stm32cubemx-mcus/when-configuring-x-cube-ai-i-get-quot-warning-this-peripheral/td-p/875672>

<https://community.st.com/t5/stm32-mcus-touchgfx-and-gui/touchgfx-designer-error-quot-unable-to-locate-c-lt-path-to/td-p/100242>